COSC1127/1125 Artificial Intelligence

School of Computing Technologies
Semester 2, 2022
Prof. Sebastian Sardiña

Tutorial No. 4
KR&R I - Propositional Logic

The aim of this tutorial is to familiarise you with propositional logic for knowledge-representation and reasoning; that is, how propositional logic can be used to represent knowledge and be used to make inferences. **The ultimate goal is that you are able to use SAT technology to solve problems, by encoding them as a SAT task. This is done in the last part of this tutorial sheet.**

You are not expected to complete all problems in the tutorial session, and you can always catch up and complete unsolved problems later.

A propositional *sentence* or *formula* is:

- A propositional letter, often written as p or q.
- $\neg P$ where P is a sentence.
- $P \wedge Q$ where P and Q are sentences.
- $P \vee Q$ where P and Q are sentences.
- $P \Rightarrow Q$ where P and Q are sentences.
- $P \Leftrightarrow Q$ where P and Q are sentences.

Sometimes we also define *exclusive or* as $P \oplus Q$ where P and Q are sentences.

We refer to $\neg, \wedge, \vee, \Rightarrow, \Leftrightarrow$ and $\oplus$ as *logical connectives*. In some places, you will find $\rightarrow$ or $\supset$ used instead of $\Rightarrow$ for implication and $\equiv$ instead of $\Leftrightarrow$. In those cases, the symbol $\Leftrightarrow$ is generally used for logical equivalence (i.e., two formulas that have exactly the same truth table).

An *interpretation* in propositional logic is a *truth assignment (or valuation)* mapping each atomic propositions to either true or false.

A *truth table* shows the truth value of formulas of interest per possible interpretation of the domain. The truth table showing the semantics (i.e., meaning) of each of the above logical connectives is as follows:

P	Q	$\neg P$	$P \wedge Q$	$P \vee Q$	$P \Rightarrow Q$	$P \Leftrightarrow Q$	$P \oplus Q$
T	T	F	T	T	T	T	F
T	F	F	F	T	F	F	T
F	T	T	F	T	T	F	T
F	F	T	F	F	T	T	F

A propositional formula is:

- **Satisfiable:** if there is at least one truth assignment that makes it true.
- **Valid or Tautology:** if every truth assignment makes it true. A formula that is valid is also satisfiable.
- **Unsatisfiable:** if there is no truth assignment that makes it true (so that all truth assignments make it false).

Two formulas P and Q are *logically equivalent* (denoted $P \equiv Q$) *iff* $P \Rightarrow Q$ and $Q \Rightarrow P$ (and hence $P \Leftrightarrow Q$) are tautologies. It means their truth value coincide on every interpretation (i.e., they have the same truth table). For example $P \vee Q$ is logically equivalent to $Q \vee P$ and to $\neg(\neg P \wedge \neg Q)$.

PART 1: Basic Logic

(a) Use truth tables to show that the following are valid (i.e. that the equivalences hold).

$P \wedge (Q \vee R) \Leftrightarrow (P \wedge Q) \vee (P \wedge R)$	Distribution of $\wedge$
$\neg(P \wedge Q) \Leftrightarrow (\neg P \vee \neg Q)$	de Morgan's Law
$\neg(P \vee Q) \Leftrightarrow (\neg P \wedge \neg Q)$	de Morgan's Law
$(P \Rightarrow Q) \Leftrightarrow (\neg Q \Rightarrow \neg P)$	Contraposition
$(P \Rightarrow Q) \Leftrightarrow (\neg P \vee Q)$	

Solution: These truth tables are very easy to find online.

(b) For each of the following sentences, decide whether it is **valid**, **unsatisfiable**, or **satisfiable but not valid**. Firstly, trying "guessing" the answer; then evaluate each properly (e.g. using truth tables). How did your guesses match up?

i. $Smoke \Rightarrow Smoke$

Solution:
Basically, just draw the truth table for each sentence - if the whole column for the sentence is T, then it is valud, if the whole column for the sentence is F, then it is unsatisfiable; neither otherwise. Eg.:

S	$S \Rightarrow S$
T	T
F	T

Therefore, it is valid.

ii. $Smoke \Rightarrow Fire$

Solution:

S	F	$S \Rightarrow F$
T	T	T
T	F	F
F	T	T
F	F	T

Satisfiable but not valid.

iii. $(Smoke \Rightarrow Fire) \Rightarrow (\neg Smoke \Rightarrow \neg Fire)$

Solution:
$A : (S \Rightarrow F)$
$B : (\neg S \Rightarrow \neg F)$

S	F	$\neg S$	$\neg F$	A	B	$A \Rightarrow B$
T	T	F	F	T	T	T
T	F	F	T	F	T	T
F	T	T	F	T	F	F
F	F	T	T	T	T	T

Satisfiable but not valid.

iv. $Smoke \vee Fire \vee \neg Fire$

Solution: Your turn!

v. $((Smoke \wedge Heat) \Rightarrow Fire) \Leftrightarrow ((Smoke \Rightarrow Fire) \vee (Heat \Rightarrow Fire))$

Solution: Your turn!

vi. $(Smoke \Rightarrow Fire) \Rightarrow ((Smoke \wedge Heat) \Rightarrow Fire)$

Exercise 1 continues on the next page...

Solution:

S	F	H	$S \Rightarrow F$	$S \wedge H$	$S \wedge H \Rightarrow F$	$(S \Rightarrow F) \Rightarrow (S \wedge H \Rightarrow F)$
T	T	T	T	T	T	T
T	T	F	T	F	T	T
T	F	T	F	T	F	T
T	F	F	F	F	T	T
F	T	T	T	F	T	T
F	T	F	T	F	T	T
F	F	T	T	F	T	T
F	F	F	T	F	T	T

This it is a validity (or tautology) as it holds in every possible interpretation.
To see it from another angle, suppose we want to prove that the implication $(S \Rightarrow F) \Rightarrow (S \wedge H \Rightarrow F)$ is true always, that is, it is a validity/tautology:

- Since it is an implication, the only possible way this is true is if $(S \Rightarrow F)$ is true, but $(S \wedge H \Rightarrow F)$ is false.
- Now for $(S \wedge H \Rightarrow F)$ to be false, $S \wedge H$ has to be true and F false.
- But if $S \wedge H$ is true, then S is true.
- Because we already assumed in the first item that $(S \Rightarrow F)$, then together with S being true, we know that F has to be true, which contradicts our second item.
- Hence, the whole implication cannot actually be made false, that is, it is a validity: always true in every possible interpretation.

vii. $Big \vee Dumb \vee (Big \Rightarrow Dumb)$

Solution:
$X : (B \vee D)$
$Y : (B \Rightarrow D)$

B	D	X	Y	$X \vee Y$
T	T	T	T	T
T	F	T	F	T
F	T	T	T	T
F	F	F	T	T

Valid.

viii. $(Big \wedge Dumb) \vee \neg Dumb$

Solution: Your turn!

(c) Consider the natural language statement "If A then B else C" (in other words, if A is true then B is true, and otherwise C is true). Give two different formulas in propositional logic that capture that statement formally.

Hints:
The "if-then" conditional statement is represented by logical implication, so "If A then B" is just $A \Rightarrow B$, have a think about how you could represent "else"

(d) Prove that $\neg(P \Rightarrow Q)$ and $\neg(\neg Q \Rightarrow \neg P)$ are all equivalent without using a truth-table

Solution: (Note: this is a partial solution)
Of course the truth table would be:

P	Q	$\neg P$	$\neg Q$	$\neg(P \Rightarrow Q)$	$P \wedge \neg Q$	$\neg(\neg Q \Rightarrow \neg P)$
T	T	F	F	F	F	F
T	F	F	T	T	T	T
F	T	T	F	F	F	F
F	F	T	T	F	F	F

Your turn! It is up to you to prove it without the truth table now.

We need to show first that if $\neg(P \Rightarrow Q)$ is true in an interpretation I, then $\neg(\neg Q \Rightarrow \neg P)$ is true in I. Then we need to show that if $\neg(\neg Q \Rightarrow \neg P)$ is true in an interpretation I, then $\neg(P \Rightarrow Q)$ is also true in I.

(e) Represent the following sentences in propositional logic. Can you prove that the unicorn is mythical? What about magical? Horned?

If the unicorn is mythical, then it is immortal, but if it is not mythical, then it is a mortal mammal. If the unicorn is either immortal or a mammal, then it is horned. The unicorn is magical if it is horned.

Solution: You can translate the sentences into the following propositional logic expressions:

1. $mythical \Rightarrow \neg mortal$
2. $\neg mythical \Rightarrow mortal \wedge mammal$
3. $\neg mortal \vee mammal \Rightarrow horned$
4. $horned \Rightarrow magical$

From statements (a) and (b), we see that if it is mythical, then it is immortal; otherwise it is a mammal. So it must be either immortal or a mammal, and thus horned. That means it is also magical. However we cannot deduce anything about whether it is mythical.

PART 2: Resolution

First, let us go over an example in full. The resolution method can be used to prove that a CNF formula (a set of clauses) is unsatisfiable, by deriving the empty clause. Consider, for example, the following CNF formula:

$$\phi = (\neg T \vee E) \wedge (T \vee \neg E) \wedge (M \vee \neg G) \wedge (\neg E \vee \neg C) \wedge (T \vee \neg M) \wedge G \wedge C$$

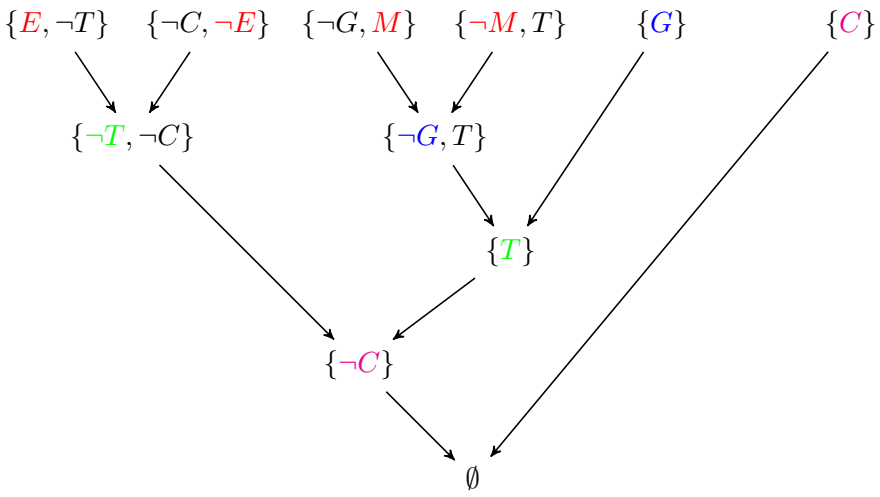
Here is a resolution proof for the empty clause:

1.	$(M \vee \neg G)$	clause in ϕ
2.	$(T \vee \neg M)$	clause in ϕ
3.	$(\neg G \vee T)$	resolution 1,2
4.	$(\neg T \vee E)$	clause in ϕ
5.	$(\neg G \vee E)$	resolution 3,4
6.	$(\neg E \vee \neg C)$	clause in ϕ
7.	$(\neg G \vee \neg C)$	resolution 5,6
8.	G	clause in ϕ
9.	C	clause in ϕ
10.	$(\neg C)$	resolution 7,8
11.	$()$	resolution 9,10

Note that clause $(T \vee \neg E)$ in ϕ was never used, so even if we drop such clause from ϕ, it still remains unsatisfiable. Also observe that, in many cases, there could be more than one (correct) resolution proof deriving the empty clause.

Exercise 2 continues on the next page...

Another way of showing a resolution proof is using a graphical representation of the proof and using sets of literals to denote clauses:



(Colours are added to show the order in which the resolutions occur: red ⇒ blue ⇒ green ⇒ magenta).

OK, now, let's do some exercises....

(a) Use propositional resolution to prove that the following sets of clauses are unsatisfiable. Observe how clauses are now represented via set notation!

i. $\{\{p, q\}, \{\neg p, r\}, \{\neg p, \neg r\}, \{p, \neg q\}\}$.

Solution:

$$\phi = (p \vee q) \wedge (\neg p \vee r) \wedge (\neg p \vee \neg r) \wedge (p \vee \neg q)$$

Here is a resolution proof for the empty clause (sets of clauses are unsatisfiable):

1.	$(p \vee q)$	clause in ϕ
2.	$(p \vee \neg q)$	clause in ϕ
3.	$(p \vee p)$	resolution 1,2
4.	$(\neg p \vee r)$	clause in ϕ
5.	$(\neg p \vee \neg r)$	clause in ϕ
6.	$(\neg p \vee \neg p)$	resolution 4,5
7.	p	resolution 3
8.	$(\neg p)$	resolution 6
9.	$()$	resolution 7,8

It can also solve using a resolution graph.

ii. $\{\{p, q\}, \{\neg r, s\}, \{\neg p, r, s\}, \{\neg q, \neg r\}, \{p, \neg s\}, \{\neg p, \neg r\}, \{r\}\}$.

Solution: Your turn!

(b) Show that the following statements are true by giving an appropriate resolution proof. You will need to convert formulas to CNF before doing a resolution proof.

i. $p \wedge (p \Rightarrow q) \models q$

Exercise 2 continues on the next page...

Solution:
Remember that $KB \models \phi$ (i.e., *KB* entails ϕ: whenever *KB* is true in an interpretation, so is ϕ) is true **if and only if** $KB \wedge \neg\phi$ is unsatisfiable. To show that $KB \wedge \neg\phi$ is unsatisfiable, we convert *KB* and $\neg\phi$ into CNF, and do a resolution proof to achieve the empty clause (i.e., a contradiction).

Here, *KB* is $p \wedge (p \Rightarrow q)$ and query is $\phi = q$. So, we want to show that the following is unsatisfiable:

$$KB \wedge \neg\phi = p \wedge (p \Rightarrow q) \wedge \neg q$$

If we convert this to CNF, we are to prove that the following formula is unsatisfiable:

$$\psi = p \wedge (\neg p \vee q) \wedge \neg q$$

Here is a resolution proof for the empty clause:

1.	$(\neg p \vee q)$	clause in ψ
2.	p	clause in ψ
3.	q	resolution 1,2
4.	$(\neg q)$	clause in ψ
5.	$()$	resolution 3,4

As we have shown that $\psi = KB \wedge \neg\phi$ is unsatisfiable, we know that ϕ must be true and $KB \models q$. ■

ii. $p \wedge (p \Rightarrow q) \wedge (q \Rightarrow r) \models r$

Solution: (<u>Note:</u> this is a partial solution)

Here, *KB* is $p \wedge (p \Rightarrow q) \wedge (q \Rightarrow r)$ and query is $\phi = r$. So, we want to show that the following is unsatisfiable:

(Your turn!)

If we convert this to CNF, we are to prove that the following formula is unsatisfiable:

(Your turn!)

Here is a resolution proof for the empty clause:

(Your turn!)

As we have shown that $\psi = KB \wedge \neg\phi$ is unsatisfiable, we know that ϕ must be true and $KB \models r$. ■

iii. $(p \Rightarrow (q \Rightarrow r)) \wedge (r \Rightarrow s) \wedge p \models \neg q \vee s$

Solution: Your turn!

iv. $(p \vee r) \wedge (p \Rightarrow (q \vee s)) \wedge (r \Rightarrow t) \models q \vee s \vee t$

Solution:
Here, *KB* is $(p \vee r) \wedge (p \Rightarrow (q \vee s)) \wedge (r \Rightarrow t)$ and query is $\phi = q \vee s \vee t$. So, we want to show that the following is unsatisfiable:

$$KB \wedge \neg\phi = (p \vee r) \wedge (p \Rightarrow (q \vee s)) \wedge (r \Rightarrow t) \wedge \neg(q \vee s \vee t)$$

If we convert this to CNF, we are to prove that the following formula is unsatisfiable:

$$\psi = (p \vee r) \wedge (\neg p \vee q \vee s) \wedge (\neg r \vee t) \wedge \neg q \wedge \neg s \wedge \neg t$$

Here is a resolution proof for the empty clause:

1.	$(p \vee r)$	clause in ψ

Exercise 2 continues on the next page...

2.	$(\neg p \vee q \vee s)$	clause in ψ
3.	$(r \vee q \vee s)$	resolution 1,2
4.	$(\neg r \vee t)$	clause in ψ
5.	$(q \vee s \vee t)$	resolution 3,4
6.	$(\neg q)$	clause in ψ
7.	$(s \vee t)$	resolution 5,6
8.	$(\neg s)$	clause in ψ
9.	t	resolution 7,8
10.	$(\neg t)$	clause in ψ
11.	()	resolution 9,10

As we have shown that $\psi = \mathit{KB} \wedge \neg\phi$ is unsatisfiable, we know that ϕ must be true and $\mathit{KB} \models q \vee s \vee t$. ■

v. $p \wedge r \wedge (p \Rightarrow q) \wedge ((q \wedge r) \Rightarrow (s \vee t)) \wedge \neg s \models t$

Solution: Your turn!

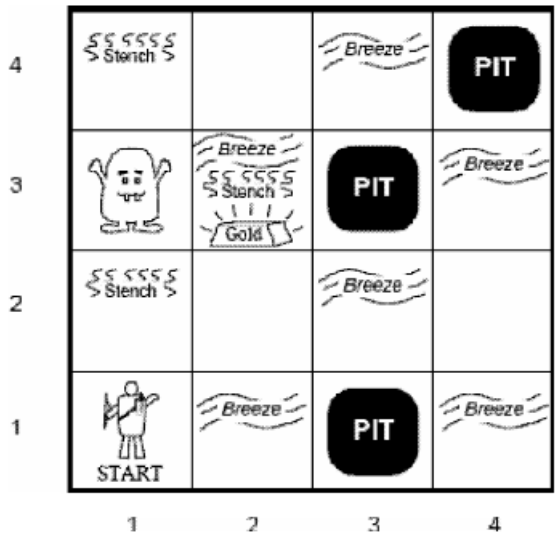
(c) Complete the truth table below. What does this tell you about resolution proofs?

P	Q	R	$\neg P \vee Q$	$P \vee R$	$(\neg P \vee Q) \wedge (P \vee R)$	$Q \vee R$	$((\neg P \vee Q) \wedge (P \vee R)) \Rightarrow (Q \vee R)$
T	T	T	**T**	**T**	**T**	**T**	**T**
T	T	F	**T**	**T**	**T**	**T**	**T**
T	F	T	**F**	**T**	**F**	**T**	**T**
T	F	F	**F**	**T**	**F**	**F**	**T**
F	T	T	**T**	**T**	**T**	**T**	**T**
F	T	F	**T**	**F**	**F**	**T**	**T**
F	F	T	**T**	**T**	**T**	**T**	**T**
F	F	F	**T**	**F**	**F**	**F**	**T**

Solution: As the formula $((\neg P \vee Q) \wedge (P \vee R)) \Rightarrow (Q \vee R)$ is valid, this means that each resolution step is correct, and hence resolution proofs are correct.

PART 3: Knowledge representation

(a) For the following Wumpus world:



i. Develop a notation capturing the important propositions.

Solution:

$$S_{xy}, W_{xy}, B_{xy}, G_{xy}, P_{xy}, Y_{xy}$$

The above denotes the sentence "there is a Stench/Wumpus/Breeze/Gold/Pit/You in square [x,y]".

ii. How would you express in a propositional logic sentence:

α) If square $[2, 2]$ has no smell then the Wumpus is not in this square or any of the adjacent squares?

Solution:

$$\neg S_{22} \Rightarrow \neg W_{22} \wedge \neg W_{21} \wedge \neg W_{12} \wedge \neg W_{32} \wedge \neg W_{23} \quad (1)$$

β) If there is stench in square $[1, 2]$ there must be a Wumpus in this square or any of the adjacent squares?

Solution:

$$S_{12} \Rightarrow W_{11} \vee W_{12} \vee W_{22} \vee W_{13} \quad (2)$$

iii. Given that the agent is standing in square [1,1] and can see (and smell) all adjacent squares, including diagonals; ; how can the agent deduce that the Wumpus is in square $[1, 3]$ using the laws of inference in propositional logic and the formulas constructed in the previous question.

Solution:

We are told that $\neg S_{22} \wedge S_{12}$

Modus Ponens and *And Elimination* applied to the above Equation 1 gives:

$$\neg W_{22} \wedge \neg W_{21} \wedge \neg W_{12} \wedge \neg W_{32} \wedge \neg W_{23}, \quad (3)$$

Modus Ponens applied to Equation 2 gives:

$$W_{11} \vee W_{12} \vee W_{22} \vee W_{13}, \quad (4)$$

Exercise 3 continues on the next page...

Unit Resolution is applied to Equation 4 based on Equation 3, and also we know the agent has been to square $[1, 1]$, hence $\neg W_{11}$. Therefore, W_{13}

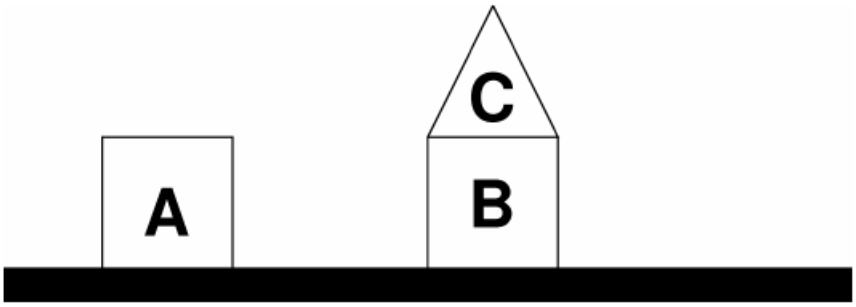
iv. How many propositions are required to fully capture the world in a single formula? What about a general world of size n?

Solution:
For a 4×4 world with 6 propositions, we need $4 \times 4 \times 6 = 96$

$$\neg S_{11} \wedge \neg W_{11} \wedge \neg B_{11} \wedge \neg G_{11} \wedge \neg P_{11} \wedge Y_{11} \wedge S_{12} \wedge \neg W_{12} \wedge \cdots \wedge P_{44} \wedge \neg Y_{44}$$

Generally, we would need $6n^2$ propositions.

(b) Represent the following "blocks world" scene in propositional calculus.



Solution: One example:

$$A_{ontable} \wedge B_{ontable} \wedge C_{onB} \wedge A_{square} \wedge B_{square} \wedge C_{triangle}$$

(c) Consider the following information available in the domain of thing encountered at sea:

1. None of the unnoticed things are mermaids.
2. Anything entered into the log is sure to be worth remembering.
3. I have never seen anything worth remembering when on a voyage.
4. Anything that is noticed is sure to be recorded in the log.

Formalise the problem in propositional logic and use resolution to formally prove that *I have never seen a mermaid at sea* using the following atomic propositions:

L: entered into the log
M: is a mermaid
S: seen by me
N: noticed
W: worth remembering

Solution:

Assertion	Implication	Clause in *KB*
1	$\neg N \Rightarrow \neg M$	$(N \vee \neg M)$
2	$L \Rightarrow W$	$(\neg L \vee W)$
3	$S \Rightarrow \neg W$	$(\neg S \vee \neg W)$
4	$N \Rightarrow L$	$(\neg N \vee L)$

"*I have never seen a mermaid at sea*" can be formally written as $(S \Rightarrow \neg M)$ or, in CNF $(\neg S \vee \neg M)$.

Take *KB* to be the conjunction of all the CNF formulas in the table above. Then, we must prove that *KB* $\wedge\neg(\neg S \vee \neg M)$ is unsatisfiable (see, the query has been negated). This is equivalent to proving that *KB* $\wedge S \wedge M$ (we will call this formula ϕ), which is in CNF, is unsatisfiable.

Here is a resolution proof for the empty clause:

1.	$(\neg S \vee \neg W)$	clause in ϕ
2.	$(W \vee \neg L)$	clause in ϕ
3.	$(\neg S \vee \neg L)$	resolution 1,2
4.	$(L \vee \neg N)$	clause in ϕ
5.	$(\neg S \vee \neg N)$	resolution 3,4
6.	$(N \vee \neg M)$	clause in ϕ
7.	$(\neg S \vee \neg M)$	resolution 5,6
8.	S	clause in ϕ
9.	$(\neg M)$	resolution 7,8
10.	M	clause in ϕ
11.	$()$	resolution 9,10

As we have shown that *KB* $\wedge S \wedge M$ is unsatisfiable, we know that the query must be true and $(S \Rightarrow \neg M)$.
■

PART 4: Knowledge bases and resolution

(a) Consider a knowledge base built of just these three weird implications:

$$\neg A \Rightarrow B$$
$$B \Rightarrow A$$
$$A \Rightarrow (C \wedge D)$$

i. Prove formula $A \wedge C \wedge D$ using Modus Ponens only, or explain why this is not possible.

Solution:
It is not possible using Modus Ponens only: Modus Ponens is not applicable to any pair of formulas in the knowledge base. An example of using Modus Ponens is:
When it is cold, I always wear my jacket $(C \Rightarrow J)$
It is cold (C)
Therefore, I am wearing my jacket (J)

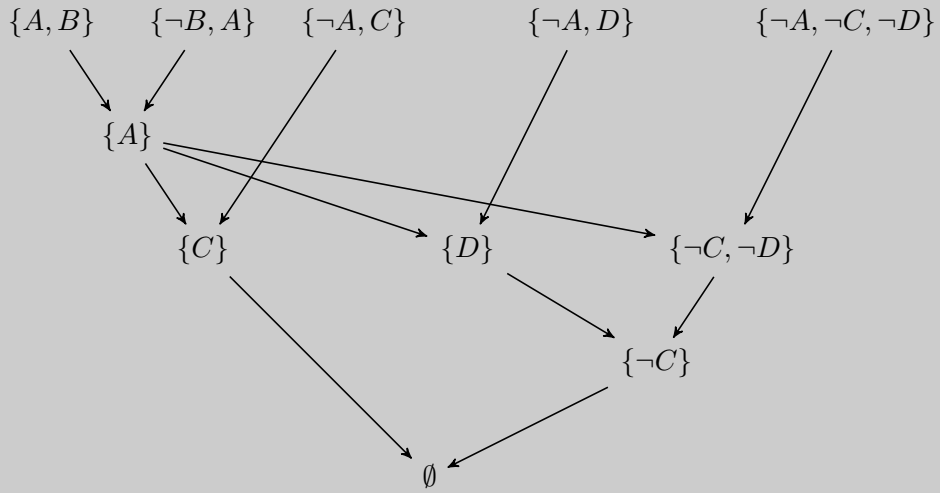
ii. Prove formula $A \wedge C \wedge D$ using resolution.

Solution:
Proof by refutation:
1. $A \vee B$ premise.
2. $\neg B \vee A$ premise.
3. $\neg A \vee C$ premise.
4. $\neg A \vee D$ premise.
5. $\neg A \vee \neg C \vee \neg D$ negated thesis.
6. A resolution 1, 2.
7. C resolution 3, 6
8. D resolution 4, 6.

Exercise 4 continues on the next page...

9. $\neg C \vee \neg D$ resolution 5, 6.
10. $\neg D$ resolution 7, 9.
11. $\perp$ resolution 8, 10

This can also be proved using a resolution tree:



Here we use the clausal set notation where set $\{P, \neg Q\}$ denotes clause $(P \vee \neg Q)$

(b) Given the following symbols and sentences:

C to indicate that Gianni is a climber;
F to indicate that Gianni is fit; L to indicate that Gianni is lucky;
E to indicate that Gianni climbs mount Everest.

i. Formalize the above sentences in propositional logic:

If Gianni is a climber and he is fit, he climbs mount Everest.
If Gianni is not lucky and he is not fit, he does not climb mount Everest.
Gianni is fit.

Solution:
$(C \wedge F) \Rightarrow E$
$(\neg L \wedge \neg F) \Rightarrow \neg E$
F

ii. Tell if the KB buit in above is consistent, and tell if some of the following sets are models for the above sentences:
$\{\}$; $\{C, L\}$; $\{L, E\}$; $\{F, C, E\}$; $\{L, F, E\}$.

(Recall that for the binary variables A, B and C; the set $S = \{A, C\}$ means A and C are true, and B is false. S is said to be a model of KB iff all statements in KB are true for that given assignment of the variables)

Solution: (<u>Note:</u> this is a partial solution)

The KB is consistent: it has at least a model, as the following check shows.

- $\{\}$ is not a model (it models 1 and 2 but not 3).
- $\{C, L\}$ Your turn!
- $\{L, E\}$ Your turn!
- $\{F, C, E\}$ is a model.
- $\{L, F, E\}$ Your turn!

(c) Prove the propositional formula $[(A \Rightarrow C) \vee (B \Rightarrow C)] \Rightarrow [(A \wedge B) \Rightarrow C]$ is valid using resolution.

Exercise 4 continues on the next page...

Solution: (Note: this is a partial solution)

We can prove this with resolution in the following way:

In the over-all implication, we can take the antecedent to be $(A \Rightarrow C) \vee (B \Rightarrow C)$ and the consequent to be $(A \wedge B) \Rightarrow C$.

Putting the antecedent into CNF gives the single clause $(\neg A \vee C \vee \neg B)$.

Putting the consequent into CNF gives $\neg(A \wedge B) \vee C \equiv \neg A \vee \neg B \vee C$.

As we are trying to prove the consequent, it must be negated: $A \wedge B \wedge \neg C$

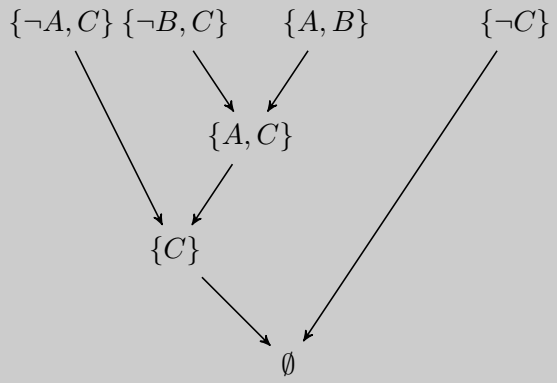
The formula can now be shown to be valid using resolution:
Your turn!

(d) Let A, B, C be propositional symbols. Given $KB = \{A \Rightarrow C, B \Rightarrow C, A \vee B\}$, tell whether C can be derived from KB or not. Use resolution.

Solution:
C can be derived with Resolution:

1. $\neg A \vee C$.
2. $\neg B \vee C$.
3. $A \vee B$.
4. $\neg C$ negated thesis
5. $B \vee C$ from 1 and 3.
6. C from 2 and 5.
7. $\{\}$ from 4 and 6.

This can also be proved using a resolution tree:



(e) Heads, I win. Tails, you lose. Use propositional resolution to prove that I always win.

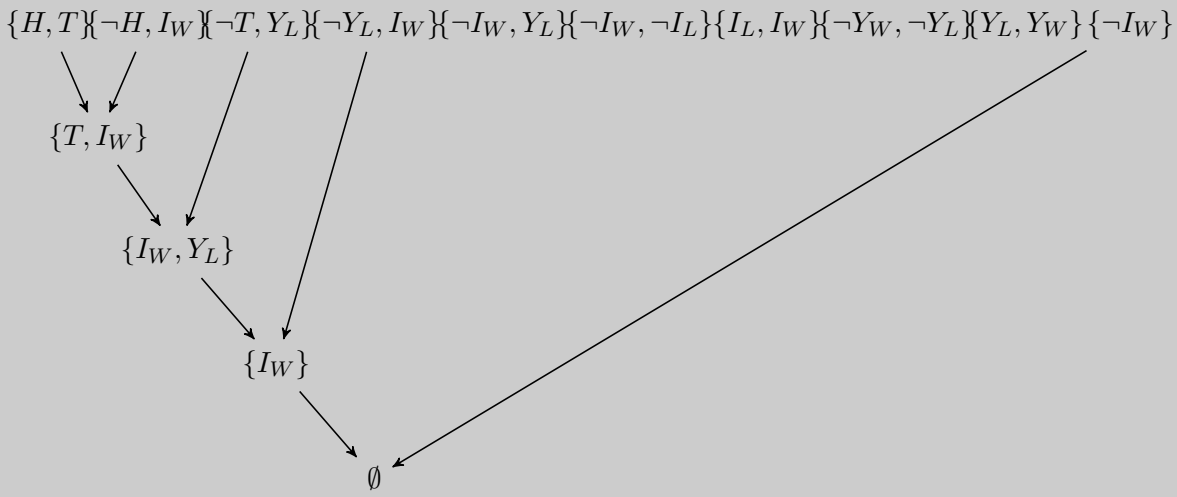
Solution: (Note: this is a partial solution)

We use H and T to signal heads or tails, resp. Also I_W and I_L to denote I win or lose, respectively, and Y_W and Y_L to denote you win or lose, resp. To formalize the problem we have:

1. I win iff I don't lose: $I_W \Leftrightarrow \neg I_L$ or $(I_W \Rightarrow \neg I_L) \wedge (\neg I_L \Rightarrow I_W)$ or $\{\{\neg I_W, \neg I_L\}, \{I_L, I_W\}\}$.
2. You win iff you don't lose: .... .
3. Zero-sum game: .... .
4. Coin either tails or heads: .... .
5. "Heads, I win": $H \Rightarrow I_W \equiv \{\neg H, I_W\}$.

6. "Tails, you lose": .... .

We need to prove that I always win, that is, that I_W is entailed by the above formulas. We convert all the above to clausal form and then do resolution with those clauses plus $\neg I_W$ and arrive to empty clause.



Here, we can see that there is a lot of redundancy in the KB. Although we have not used all of the clauses, so long as we are able to derive a contradiction, the resolution is valid.

PART 5: SAT Technology

Satisfiability in propositional logic (named SAT) is a very active area of research and used in the industry, for constraint satisfaction, optimisation, planning, etc.

(a) Research and explain what is the DIMACS format used as a standard for specifying CNF formulas (and used by most SAT solvers). Check, for example this and this.

Solution:

- The CNF file format is an ASCII file format.
- The file may begin with comment lines. The first character of each comment line must be a lower case letter "c". Comment lines typically occur in one section at the beginning of the file, but are allowed to appear throughout the file.
- The comment lines are followed by the "problem" line. This begins with a lower case "p" followed by a space, followed by the problem type, which for CNF files is "cnf", followed by the number of variables followed by the number of clauses.
- The remainder of the file contains lines defining the clauses, one by one.
- A clause is defined by listing the index of each positive literal, and the negative index of each negative literal. Indices are 1-based, and for obvious reasons the index 0 is not allowed.
- The definition of a clause may extend beyond a single line of text. The definition of a clause is terminated by a final value of "0".
- The file terminates after the last clause is defined.

(b) Get a SAT solver (e.g., MiniSAT SAT; SAT4j, file `sat4j.jar`; or Microsoft Z3 Prover), and test it on some CNF files in DIMACS format from here.

Solution: The official MiniSAT solver can be found here. Binaries are available for Linux and Windows under Cywin.

The SAT4j solver is Java-based so it runs on any platform, Linux, Mac, or Windows. You just need, of coure, Java JRE installed.

Exercise 5 continues on the next page...

What is more, you can even check it with an *online* version of the Cryptominisat solver. Just paste the content of your DIMACS file there and click Ready on the upper right corner!

(c) Finally, solve Part 3.c and ALL the problems in PART 4 by making use of a SAT solver.

Solution: Ultimately, we are to build a DIMACS file encoding the problem as a SAT task.
Next we only the detail the solution for the **sea domain** problem (Part 3.(c)):

```
p cnf 5 6
1 -2 0
-3 4 0
-5 -4 0
-1 3 0
5 0
2 0
```

Now, with this at hand, can you map the DIMACS variables to the actual propositional letters used in the problem?
When ran on SAT4j, you get:

```
[ssardina@Thinkpad-X1 SAT]$ java -jar sat4j.jar MiniSAT sea-domain.cnf
c SAT4J: a SATisfiability library for Java (c) 2004 Daniel Le Berre
c version 1.0.290RC
c org.sat4j.minisat.core.Solver@f6f4d33
c solving tute-07-sea-domain.cnf
c reading problem time 0.012
c #vars     5
c #clauses  4
s UNSATISFIABLE
c Total CPU time (ms) : 0.015
```

When ran in this online SAT solver, the output is (check the word `UNSATISFIABLE`):

```
c CryptoMiniSat version 5.6.8
c CryptoMiniSat SHA revision 0acd0961307384074716139 6ac47abf35ec29320
c -- header says num vars:              3
c -- header says num clauses:           5
c -- clauses added: 6
c -- xor clauses added: 0
c -- vars added 5
s UNSATISFIABLE
c ------- FINAL TOTAL SEARCH STATS ---------
c restarts                 : 0          (0.00      confls per restart)
c blocked restarts         : 0          (0.00      per normal restart)
c decisions                : 0          (0.00      % random)
c propagations             : 0
c decisions/conflicts      : 0.00
c conflicts                : 0
c conf lits non-minim      : 0          (0.00      lit/confl)
c conf lits final          : 0.00
c cache hit re-learnt cl   : 0          (0.00      % of confl)
c red which0               : 0          (0.00      % of confl)
c props/decision           : 0.00
c props/conflict           : 0.00
c 0-depth assigns          : 5          (100.00    % vars)
```


Exercise 5 continues on the next page...

```
c [occ-substr] long subBySub: 0 subByStr: 0 lits-rem-str: 0
c [scc] new: 0 BP 0M  T: 0.00
c Conflicts in UIP          : 0
c Mem used                  : 0.00         MB
```

Please check this video that explains this solution in more detail.

End of tutorial worksheet

COSC1127/1125 Artificial Intelligence

School of Computing Technologies
Semester 2, 2022
Prof. Sebastian Sardiña
Tutorial No. 5
KR&R II - First Order Logic

For some of the questions below, you may want to check the great slides by Torsten Hahmann (CSC 384 at University of Toronto) on Skolemization, Most General Unifiers, First-Order Resolution. It includes the Skolemization process to remove existential quantification, the algorithm for finding MGUs, and FOL resolution; all with great fully detailed examples!

Predicates

A *term* is either a variable or a name (sometimes called a constant).[1]

A *formula* (or first-order formula) is
- $p(x_1, \ldots, x_n)$ where $x_1, \ldots, x_n$ are terms. p is called a *predicate*, and $p(x_1, \ldots, x_n)$ is called an *atom*.
- $\neg P$ where P is a formula.
- $P \vee Q$, $P \wedge Q$, $P \Rightarrow Q$ and $P \Leftrightarrow Q$ where P and Q are formulae.
- $\forall x P$ and $\exists x P$ where P is a formula with free variable x. $\forall$ and $\exists$ are called *quantifiers*.

A *literal* is an atom or the negation of an atom. For example, $p(1, 2)$ and $\neg q(3)$ are both literals, but $\forall x \neg p(x)$ is not. We will say a formula is in *literal form* if all negations in the formula occur only in literals. For example, the formula $\neg \forall x\ p(x, x)$ is not in literal form, whereas $\neg p(1, 2)$ is in literal form.

Note that all the previous logical equivalences (denoted $\equiv$) are still true. The ones from last week and some specific to quantifiers are below.

$P \wedge (Q \vee R) \equiv (P \wedge Q) \vee (P \wedge R)$	*Distributive Law 1*
$P \vee (Q \wedge R) \equiv (P \vee Q) \wedge (P \vee R)$	*Distributive Law 2*
$\neg\neg P \equiv P$	
$\neg(P \wedge Q) \equiv (\neg P) \vee (\neg Q)$	*De Morgan Law 1*
$\neg(P \vee Q) \equiv (\neg P) \wedge (\neg Q)$	*De Morgan Law 2*
$\forall x\ p(x) \equiv \forall y\ p(y)$	$\exists x\ p(x) \equiv \exists y\ p(y)$
$\forall x\ \forall y\ P \equiv \forall y\ \forall x\ P$	$\exists x\ \exists y\ P \equiv \exists y\ \exists x\ P$
$\neg \forall x\ P \equiv \exists x\ \neg P$	$\neg \exists x\ P \equiv \forall x\ \neg P$
$\forall x\ P(x) \Rightarrow Q(x) \equiv \forall x \neg Q(x) \Rightarrow \neg P(x)$	Contrapositive

Please remember that $\equiv$ is NOT a logical symbol, so $\alpha \equiv \beta$ is NOT a formula in the FOL language. Instead, $\alpha \equiv \beta$ is a shorthand for the claim (at the English meta-level), that $\alpha \models \beta$ and $\beta \models \alpha$, or what is the same, that for every FOL interpretation/models M, $M \models \alpha$ if and only if $M \models \beta$.

[1] Strictly speaking there are other kinds of terms as well, but we will not be concerned with them in this course.

Using quantifiers and connectives

The choice of quantifiers and connectives has a very big impact on the ultimate meaning of a logical statement. For example, if X is an object in the universe of discourse (i.e., the set of objects being considered), the predicate $zebra(X)$ means X is a zebra and the predicate $striped(X)$ means X is striped, the statement

$$\forall x\ zebra(x) \Rightarrow striped(x),$$

is read as *all zebras are striped*. Intuitively, you can think of logical implication as an *if-then* conditional statement, in which case this is read as *for all x in the universe of discourse, if x is a zebra, then x is striped*.

The meaning of this statement can be changed by changing the connective. For example, the statement

$$\forall x\ zebra(x) \wedge striped(x),$$

is read as *everything is a zebra and everything is striped*, meaning everything in the universe of discourse is a striped zebra!

Similarly, changing the quantifier will also change the meaning. For example, the statement

$$\exists x\ zebra(x) \wedge striped(x),$$

is read as *there exists an x, where x is a zebra and x is striped*, meaning that out of all of the objects in the universe of discourse *at least one* is a striped zebra. Finally, the statement

$$\exists x\ zebra(x) \Rightarrow striped(x),$$

is a bit more tricky and you will almost never see this combination of quantifier and connective. This statement is read as *there exists an x, where if x is a zebra, then x is striped*. The difference is a bit subtle, but the important thing to understand is that this statement only guarantees that x exists, it does not guarantee that x is a zebra or that it is striped!

As a general rule-of-thumb, the universal quantifier ($\forall x$) is usually paired with the implication connective ($\Rightarrow$) and the existential quantifier ($\exists x$) is usually paired with the conjunction connective ($\wedge$).

PART 1: Predicate logic formulas

(a) Choose a vocabulary of predicates, constants and functions appropriate for representing the following information in First Order Logic, then represent each sentence in FOL.

i. "There is someone who is loved by everybody"

Solution:
$(\exists y)(\forall x) loves(x, y)$

ii. "All cats are lazy"

Solution: (Note: this is a partial solution)
$....(cat(x) \Rightarrow ....)$

iii. "Some students are clever"

Solution:
$(\exists x)(student(x) \wedge clever(x))$

iv. "No student is rich"

Solution:
$(\neg\exists x)(student(x) \wedge rich(x))$
or
$(\forall x)(student(x) \Rightarrow \neg rich(x))$

Exercise 1 continues on the next page...

v. "Every man loves a woman" (represent both meanings)

Solution: Your turn!

vi. "Everything is bitter or sweet"

Solution:
$(\forall x)(bitter(x) \vee sweet(x))$

vii. "Everything is bitter or everything is sweet"

Solution:
$(\forall x)bitter(x) \vee \forall y sweet(y)$

viii. "Martin has a new bicycle"

Solution: Your turn!

ix. "Lynn gets a present from John, but she doesn't get any present from Peter"

Solution: $(\exists x)(present(x) \wedge gets(Lynn, x, John)) \wedge (\neg \exists y)(present(y) \wedge gets(Lynn, y, Peter))$

(b) Do the same as for the previous question.

i. Anyone who is rich is powerful.

Solution:
$(\forall x)(rich(x) \Rightarrow powerful(x))$

ii. Anyone who is powerful is corrupt.

Solution: Your turn!

iii. Anyone who is meek and has a corrupt boss is unhappy.

Solution:
$(\forall x)(\exists y)(meek(x) \wedge isBoss(y, x) \wedge corrupt(y) \Rightarrow unhappy(x))$

iv. Not all students take both Computing-Theory and OO-Programming.

Solution: Your turn!

v. Only one student failed Computing-Theory.

Solution:
$(\exists x)student(x) \wedge fails(x, CT) \wedge (\forall y)student(y) \wedge fails(y, CT) \Rightarrow x = y$
In this example, an **equality symbol** is used to make a statment about the effect that two terms refer to the same object. For the above first sentence,

$$(\exists x)student(x) \wedge fails(x, CT) \wedge (\forall y)student(y) \wedge fails(y, CT)$$

would only assert that all instances of y and there is at least one x that failed CT, but nothing says that x and y are referring to the same object.
You can imagine that you run a loop going over all possible y values, and there is an x value where $x = y$ is true. When this is the case, the implication stands.
Note that in a knowledge base, the existential quantifier is eliminated by replacing it with specific instances. Based on this assumption, the above sentence will refer to only one specific student who failed CT.

vi. Only one student failed both Computing-Theory and OO-Programming.

Solution: Your turn!

vii. The best score in Computing-Theory was better than the best score in OO-Programming.

Exercise 1 continues on the next page...

Solution:
$(\exists x)(score(x, CT) \wedge ((\forall y)score(y, OO) \Rightarrow better(x, y)))$
Since there is no negation in front of the universal quantifier (and no scope clashes), it can be moved to the front of the formula without changing the expression.
$(\exists x)(\forall y)(score(x, CT) \wedge (score(y, OO) \Rightarrow better(x, y)))$

viii. Every person who dislikes all donkeys is smart.

Solution:
$(\forall x)person(x) \wedge ((\forall y)donkey(y) \Rightarrow dislikes(x, y)) \Rightarrow smart(x)$

ix. There is a woman who likes all men who are not footballers.

Solution: Your turn!

x. There is a barber who shaves all men who do not shave themselves.

Solution:
$(\exists x)barber(x) \wedge ((\forall y)man(y) \wedge \neg shaves(y, y)) \Rightarrow shaves(x, y)$

xi. No person likes a lecturer unless the lecturer is smart.

Solution: Your turn!

xii. Politicians can fool some of the people all of the time, and they can fool all of the people some of the time, but they can't fool all of the people all of the time.

Solution:
$(\forall x)politician(x) \Rightarrow$
$((\exists y)(\forall t)person(y) \wedge fools(x, y, t)) \wedge$
$((\exists t)(\forall y)person(y) \wedge fools(x, y, t)) \wedge$
$\neg((\forall t)(\forall y)person(y) \Rightarrow fools(x, y, t))$

(c) Consider the following story:

> I married a widow (let's call her w) who has a grown up daughter (d). My father (f), who visited us quite often, fell in love with my step-daugher and married her. Hence my father became my son-in-law and my step-daughter became my mother. Some months later, my wife gave birth to a son (s_1), who became the brother-in-law of my father, as well as my uncle. The wife of my father, that is, my step-daughter, also had a son (s_2).

Using predicate calculus, create a set of expressions that represent the situation in the above story. Extend the kinship relations to in-laws and uncle. Use generalised modus ponens to prove 'I am my own grandfather'.

Solution:
This works fine (with a little violation of the accepted semantics for family relationships).
Here is an example showing how we can prove this; we can have the following rules (among others):

$$(\forall x)(\forall y)child(x, y) \equiv parent(y, x) \quad (1)$$
$$(\forall x)(\forall y)(\exists z)grandparent(x, y) \equiv parent(x, z) \wedge parent(z, y) \quad (2)$$

From the given story we can get the following facts (once again, among others):

$$parent(F, I) \text{ (since } F \text{ is my father)}, \quad (3)$$

$$child(F, I) \text{ (since my father becomes my son-in-law)}. \quad (4)$$

Some of the other rules are: $(parent(W, D), spouse(I, W), spouse(F, D)$, etc), but we don't really need them since we can already prove our conclusion.

From 1 and 4 (take $y = F$ and $x = I$) we derive that

$$parent(I, F) \tag{5}$$

Finally, from 3 and 5 we derive (take $x = I, y = I$ and $z = F$):

$$grandparent(I, I)$$

Thus, "I am my own grandfather"!

PART 2: Representing family relationships

You are given an enormous table of family information. This consists of a very large number of facts, which are organised into predicates as below.

Predicate	**Meaning**
$parent(x, y)$	x is a parent of y
$male(x)$	x is male
$female(x)$	x is female

Use this information to define the relationships below by giving an appropriate formula based on the above three predicates.

Example: To capture he relation "x is the brother of y", we can write:

$$male(x) \wedge \exists z(parent(z, x) \wedge parent(z, y))$$

and so define the new relation $brother(x, y)$ via the following formula (note the formula is an equivalence between the new predicate and the formula):

$$\forall x, y[brother(x, y) \Leftrightarrow (male(x) \wedge \exists z(parent(z, x) \wedge parent(z, y))]$$

This formula is, basically, defining what the brother relation is in terms of the other two relations. Note the universal quantification at the outset of the whole equivalence formula: the relation between x and y holds for every pair of x and y.

(a) x is the sister of y.

Solution: $female(x) \wedge \exists z\ parent(z, x) \wedge parent(z, y)$
and if we want to define a new relation to capture the *sister* notion we can write the following formula:

$$\forall x, y[sister(x, y) \Leftrightarrow female(x) \wedge \exists z\ parent(z, x) \wedge parent(z, y)]$$

(b) x is the uncle of y.

Solution: Your turn!

(c) x is the grandmother of y.

Solution: $female(x) \wedge \exists w\ parent(x, w) \wedge parent(w, y)$
and if we want to define a new relation to capture the *grandmother* notion:

Exercise 2 continues on the next page...

$$\forall x, y[grandma(x, y) \Leftrightarrow female(x) \wedge \exists w\ parent(x, w) \wedge parent(w, y)]$$

(d) x is an orphan.

Solution: Your turn!

(e) x is not an orphan.

Solution: $\exists z\ parent(z, x)$
If we had a predicate $orphan(x)$ already we can state when exactly that predicate is false as follows:

$$\forall x[\neg orphan(x) \Leftrightarrow \exists z\ parent(z, x)]$$

While not necessary, on could use a different predicate to state that x is not an orphan, something like this:

$$\forall x[notOrphan(x) \Leftrightarrow \exists z\ parent(z, x)]$$

See that $notOrphan(x)$ is just another predicate name. The fact that I use the English "not" at the start does not imply it is the negation of anything. I could have named the new predicate $hello(x)$ if I wanted.
Now, if we had defined $orphan(x)$ as in the previous question, one would be able to formally prove that the formula $\forall x(\neg orphan(x) \Leftrightarrow notOrphan(x))$ is a indeed tautology!

(f) x is someone's mother.

Solution: $female(x) \wedge \exists z\ parent(x, z)$
and if we want to capture it in a new predicate, we can use the following formula:

$$\forall x[isMother(x) \Leftrightarrow female(x) \wedge \exists z\ parent(x, z)]$$

(g) x is someone's son.

Solution: Your turn!

(h) x is a first cousin of y, that is, x and y have a common grandparent.

Solution: $\exists z \exists w \exists v\ parent(w, x) \wedge parent(v, y) \wedge (w \neq v) \wedge parent(z, w) \wedge parent(z, v)$
and if we want to capture it in a new predicate, we can use the following formula:

$$\forall x, y[cousin(x, y) \Leftrightarrow \exists z \exists w \exists v\ parent(w, x) \wedge parent(v, y) \wedge (w \neq v) \wedge parent(z, w) \wedge parent(z, v)]$$

(i) x is a first or second cousin of y.

Solution:

$$\begin{aligned}&(\exists z \exists w \exists v\ parent(w, x) \wedge parent(v, y) \wedge (w \neq v) \wedge parent(z, w) \wedge parent(z, v)) \vee \\ &(\exists z \exists w \exists v \exists u \exists t\ parent(w, x) \wedge parent(v, y) \wedge parent(u, w) \wedge parent(t, v) \wedge \\ &(u \neq t) \wedge parent(z, u) \wedge parent(z, t))\end{aligned}$$

and if we want to capture it in a new predicate, we can use the following formula:

Exercise 2 continues on the next page...

$$\forall x, y[fs\_cousin(x, y) \Leftrightarrow (\exists z \exists w \exists v\ parent(w, x) \wedge parent(v, y) \wedge (w \neq v) \wedge parent(z, w) \wedge parent(z, v)) \vee (\exists z \exists w \exists v \exists u \exists t\ parent(w, x) \wedge parent(v, y) \wedge parent(u, w) \wedge parent(t, v) \wedge (u \neq t) \wedge parent(z, u) \wedge parent(z, t))]$$

(j) x is an ancestor of y.

Solution: This can only be done by defining the relationship "recursively", i.e. in term of itself:

$$\forall x, y[ancestor(x, y) \Leftrightarrow (parent(x, y) \vee \exists z(parent(x, z) \wedge ancestor(z, y)))]$$

However, it is important to note that this formula would actually not work in predicate logic, that is, it will not correctly define the ancestor notion in terms of the parent one. This is because FOL/predicate logic is not expressive enough to capture Transitive Closure (but this is out of the scope of the course).

(k) x is a descendant of y.

Solution: Your turn!

(l) All of x's grandchildren are male.

Solution: $\forall y \forall z(parent(x, y) \wedge parent(y, z)) \Rightarrow male(z)$

(m) x does not have any grandparents.

Solution: $\neg \exists y \exists z\ parent(y, x) \wedge parent(z, y)$

If you are keen, you can try executing some similar definitions using the language *Prolog*. Download and install a Prolog interpreter (a good one is SWI-Prolog), and load the file *family.pl* available here. The syntax may take a little getting used to, but in this file you will find various definitions like those in the previous question.

PART 3: Unification

(a) Give the most general unifier (MGU) for the following pairs of expressions, or say why it does not exist. In all of these expressions variables are lower-case x, y, or z.

i. $p(a, b, b), p(x, y, z)$.

Solution:
To find the MGU, we use the algorithm described here.

$$\begin{aligned} S_0 &= \{p(a, b, b), p(x, y, z)\} \\ D_0 &= \{a, x\} && \text{[first disagreement set]} \\ \sigma &= \{x/a\} \\ S_1 &= \{p(a, b, b), p(a, y, z)\} \\ D_1 &= \{b, y\} && \text{[second disagreement set]} \\ \sigma &= \{x/a, y/b\} \\ S_2 &= \{p(a, b, b), p(a, b, z)\} \\ D_3 &= \{b, z\} && \text{[third disagreement set]} \\ \sigma &= \{x/a, y/b, z/b\} \\ S_3 &= \{p(a, b, b), p(a, b, b)\} \end{aligned}$$

Exercise 3 continues on the next page...

No disagreement, MGU found! $\sigma = \{x/a, y/b, z/b\}$.
Observe that $\sigma' = \{x/a, y/b, z/y\}$ is not an MGU because it is not even a unification of both structures. In fact, $p(x,y,z)\sigma = p(a,b,b) \neq p(x,y,z)\sigma' = p(a,b,y)$.

ii. $q(y, g(a,b)), q(g(x,x), y)$.

Solution:

$$\begin{aligned} S_0 &= \{q(y, g(a,b)), q(g(x,x), y)\} \\ D_0 &= \{y, g(x,x)\} && \text{[first disagreement set]} \\ \sigma &= \{y/g(x,x)\} \\ S_1 &= \{q(g(x,x), g(a,b)), q(g(x,x), g(x,x))\} \\ D_1 &= \{g(a,b), g(x,x)\} && \text{[second disagreement set]} \end{aligned}$$

No unifier (x cannot bind to both a and b).

iii. $older(father(y), y), older(father(x), john)$.

Solution: Your turn!
If you still need additional guidance, please check this video. However, note that if you needed to rely on the video, then there was probably insufficient learning. The best & expected case would be that you use the video to verify your approach.

iv. $knows(father(x), y), knows(x, x)$.

Solution:

$$\begin{aligned} S_0 &= \{knows(father(x), y), knows(x, x)\} \\ D_0 &= \{father(x), x\} && \text{[first disagreement set]} \end{aligned}$$

No unifier, because the occurs-check prevents unification of x with $father(x)$

v. $r(f(a), b, g(f(a))), r(x, y, z)$.

Solution: Your turn!

vi. $older(father(y), y), older(father(y), john)$.

Solution:

$$\begin{aligned} S_0 &= \{older(father(y), y), older(father(y), john)\} \\ D_0 &= \{y, john\} && \text{[first disagreement set]} \\ \sigma &= \{y/john\} \\ S_1 &= \{older(father(john), john), older(father(john), john)\} \end{aligned}$$

No disagreement, MGU found! $\sigma = \{y/john\}$.

vii. $f(g(a, h(b)), g(x,y)), f(g(z,y), g(y,y))$.

Solution: Your turn!

(b) Some more (y, w, z, v, u are all variables):

i. $p(f(y), w, g(z)), p(v, u, v)$.

Solution:
Impossible as v has to match both $f(y)$ and $g(z)$ and f and g are different symbols.

ii. $p(f(y), w, g(z)), p(v, u, x)$.

Exercise 3 continues on the next page...

Solution: Your turn!

iii. $p(a, x, f(g(y))), p(z, h(w), f(w))$.

Solution:
$\{z/a, x/h(w), w/g(y)\}$

iv. $p(z, h(w), g(z)), p(v, u, v)$.

Solution: Your turn!

v. $p(a, h(w), f(g(y))), p(z, x, f(w))$.

Solution:
$\{z/a, x/h(w), w/g(y)\}$

PART 4: Representation and reasoning

(a) Write down logical representations for the following sentences, suitable for use with generalised modus ponens. Pay special attention to implications and quantification.

i. Horses, cows and pigs are mammals.

Solution:
$\forall x[Horse(x) \Rightarrow Mammal(x)]$
$\forall x[Cow(x) \Rightarrow Mammal(x)]$
$\forall x[Pig(x) \Rightarrow Mammal(x)]$

ii. An offspring of a horse is a horse.

Solution:
$\forall x, y[Offspring(x, y) \wedge Horse(y) \Rightarrow Horse(x)]$

iii. Bluebeard is a horse.

Solution:
$Horse(bluebeard)$

iv. Bluebeard is Charlie's parent.

Solution: Your turn!

v. Offspring and parent are inverse relations.

Solution:
$\forall x, y[Offspring(x, y) \Rightarrow Parent(y, x)]$
$\forall x, y[Parent(x, y) \Rightarrow Offspring(y, x)$

vi. Every mammal has a parent.

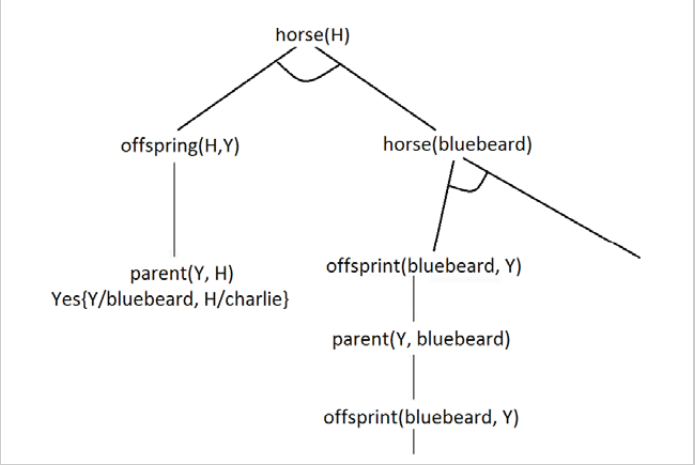
Solution: Your turn!

(b) Using the logical expressions from the previous question do the following:

i. Draw the proof tree using backward chaining (that is, starting from your query) for the query "there exists a horse" - $(\exists x)Horse(x)$.

Solution:
The proof tree is shown below. The branch with $Offspring(bluebeard, y)$ and $Parent(y, bluebeard)$ repeats indefinitely, so the rest of the proof is never reached.

Exercise 4 continues on the next page...



ii. Repeat the proof using forward chaining (that is, start with the facts and derive further conclusions). Do you get the the same results as before?

Solution: Your turn!

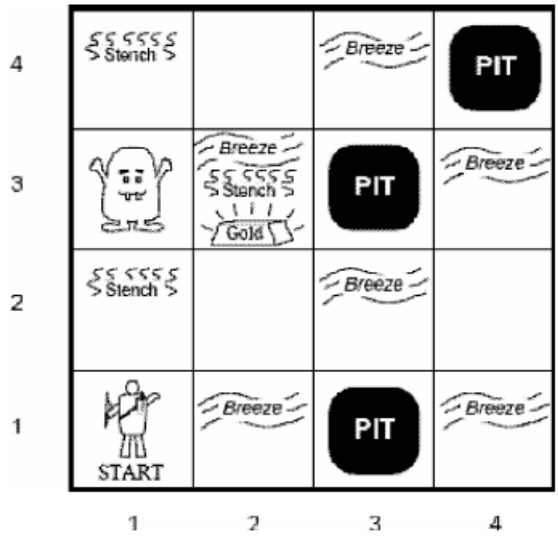
iii. How many solutions for H are there?

Solution:
Both $bluebeard$ and $charlie$ are horses.

(c) A popular children's riddle is *"Brothers and sisters I have none, but that man's father is my father's son"*. Use the rules of the kinship domain to work out who that man is.

Solution:
The son (of the man speaking).

(d) For the following Wumpus world:



i. Develop a notation capturing the important aspects of this domain in first order logic.

Solution: (<u>Note:</u> this is a partial solution)
Similar to previous tutorial for propositional logic but now stated using first order.

ii. How would you express in a first order logic sentence:

α) If a square has no smell then the Wumpus is not in this square or any of the adjacent squares?

Solution:
$\forall x[\neg Smell(x) \Rightarrow \forall y((Adjacent(x,y) \vee x = y) \Rightarrow \neg WumpusAt(y))]$.

β) If there is stench in a square there must be a Wumpus in this square or in one of the adjacent squares?

Solution:
$\forall x[Smell(x) \Rightarrow \exists y((Adjacent(x,y) \vee x = y) \wedge WumpusAt(y))]$.

iii. Suppose the agent has traversed the path $(1,1) \Rightarrow (1,2) \Rightarrow (2,2) \Rightarrow (1,2)$. How can the agent deduce that the Wumpus is in square [1,3] using the laws of inference of first order logic?

Solution: It follows from (β) above and the fact that the knowledge base includes or entails:

$Adjacent((1,3),(1,2))$
$Smell((1,2))$
$\neg WumpusAt((1,1))$
$\neg Smell((2,2))$(or directly $\neg WumpusAt((2,2))$)

PART 5: Resolution proofs

As with propositional logic, applying resolution to first-order logic formulas requires that they be first converted to conjunctive normal form. This is a bit more complicated when dealing with predicates however, because of the existential and universal quantifiers. We need to find a certain kind of reduction from first-order logic to propositional logic, in order to use the principles of resolution to prove our statements, and Herbrand's theorem provides us with such a reduction. However, this theorem applies to the domain closure of a set of formulas with no quantifier, so in order to apply it, we need a way to remove quantifiers from first-order logic forumulas.

Basically, a first-order clause (disjunction of literals) is always assumed to be universally quantified. So, if a clause like $(P(x) \vee \neg Q(x,y) \vee R(y))$ stands for the sentence $\forall x,y.(P(x) \vee \neg Q(x,y) \vee R(y))$. So, when the only quantifiers we have are universal, this is easy: if we have a sentence $\forall x.P(x) \vee Q(y)$, its clausal representation is just $(P(x) \vee Q(y))$. Similarly, $\forall x.P(x) \Rightarrow Q(x)$ can be written in CNF as $(\neg P(x) \vee Q(x))$.

Where it becomes more complicated is when we are using the existential quantifier. If we have a formula $\exists x.P(x)$, we cannot just write clause $P(x)$, sa that clause stands for $\forall x.P(x)$. This is where the idea of *Skolemisation* comes in. As $\exists x.P(x) \vee Q(x)$ says that "there exists some value of x" for which $P(x) \vee Q(x)$ is true, we can assume a "witness" value representing such value, and hence use a constant A, so-called the *Skolem constant* for such x, for this value and write $P(A) \vee Q(A)$.

Now, what happens with a formula like $\forall x, \exists y.(P(x) \wedge Q(y))$? The fact is that the value of y that makes the formula true, will depend on each specific x. So, when dealing with formulas that mix quantifiers, we need to use a so-called *Skolem function*, to indicate that the element we are claiming to exist depend on the value of the previously universally quantified variables. A Skolem function is an n-place function, where n is the number of universal quantifiers that appear before the existentially quantified variable.

For example:

$$[\forall x.P(x) \Rightarrow \exists y.Q(y) \wedge R(x,y)] \equiv [P(x) \Rightarrow Q(f(x)) \wedge R(x,f(x))]$$
$$[\forall x \forall y.P(x,y) \Rightarrow \exists z.Q(z) \wedge R(x,z)] \equiv [P(x,y) \Rightarrow Q(f(x,y)) \wedge R(x,f(x,y))]$$
$$[\forall x.P(x) \Rightarrow \exists y \forall z.(Q(y) \wedge R(y,z))] \equiv [P(x) \Rightarrow (Q(f(x)) \wedge R(f(x),z))]$$
$$[\exists x \forall y \forall z.P(x) \wedge (Q(y) \Rightarrow R(x,z))] \equiv [P(A) \wedge (Q(y) \Rightarrow R(A,z))]$$

In this last example, we use the Skolem constant A, as there is no universal quantifier that occurs before the existential quantifier. Of course, these examples are only used to demonstrate how to remove the quantifiers from these formulas; in order to apply resolution, we must also convert them into CNF.

Exercise 5 continues on the next page...

When doing KRR below, we take information given in English and represent such information in first-order logic. We then suitably convert the resulting formulas so we can perform resolution, by transforming the formulas to CNF + Skolemization. Finally, we construct a resolution proof for the query of concern. There are fully work examples in the book (Section 9.5 - Resolution).

(a) Consider the following sentences:

1. Marcus was a man.
2. Marcus was a Roman.
3. All men are people.
4. Caesar was a ruler.
5. All Romans were either loyal to Caesar or hated him (or both).
6. Everyone is loyal to someone.
7. People only try to assassinate rulers they are not loyal to.
8. Marcus tried to assassinate Caesar.

Carry out the following tasks:

i. Convert each of these sentences into first-order logic and then convert each formula into clausal form. Indicate any Skolem functions or constants used in the conversion.

Solution: Check **??**this video for a solution explanation of the part.
First-order sentences:

1. $Man(marcus)$
2. ....
3. $\forall x.Man(x) \Rightarrow Person(x)$.
4. $Ruler(caesar)$.
5. $\forall x.Roman(x) \Rightarrow Loyal(x, caesar) \vee Hate(x, caesar)$.
6. $\forall x.\exists y.Loyal(x, y)$
7. ....
8. $TryKill(marcus, caesar)$.

ii. Convert the negation of the question "Who hated Caesar?" into causal form (with an answer literal)

Solution:
Conversions to CNF:

1. $Man(marcus)$
2. $Roman(marcus)$
3. $\neg Man(x) \vee Person(x)$.
4. .... .
5. $\neg Roman(x) \vee Loyal(x, caesar) \vee Hate(x, caesar)$.
6. $Loyal(x, f(x))$
   (as the existential quantifier $\exists y$ is within the scope of the universal quantifier $\forall x$, a Skolem function $f(x)$ must be substituted for the variable y in order to remove the existential quantifier)
7. $(\neg TryKill(x, y) \vee Ruler(y))$ and .... .
8. $TryKill(marcus, caesar)$.

iii. Consider the query question "Who hated Caesar?". First, express it in FOL. Then, convert the negation of the question into causal form (with an answer literal)

Solution: Query: $\exists z.Hate(z, caesar)$.
Query with answer predicate: $\exists z.[Hate(z, caesar) \wedge \neg A(z)]$.
Negation of query with answer predicate: $\neg Hate(z, caesar) \vee A(z)$

iv. Derive the answer (to the question "Who hated Caesar?") using first-order resolution. Give the answer

Exercise 5 continues on the next page...

in English. In the proof use the notation developed in lectorials.

Solution: (Note: this is a partial solution)
Query: $\exists z.Hate(z, caesar)$.

Query with answer predicate: $\exists z.[Hate(z, caesar) \wedge \neg A(z)]$.

Negation of query with answer predicate: $\neg Hate(z, caesar) \vee A(z)$

The resolution follows easily until we get a clause $A(marcus)$: Marcus hated Caesar.

1.	$\{Man(marcus)\}$	clause from KB
2.	$\{Roman(marcus)\}$	clause from KB
3.	$\{\neg Man(x), Person(x)\}$	clause from KB
4.	$\{Ruler(ceasar)\}$	clause from KB
5.	$\{\neg Roman(x), Loyal(x, caesar), Hate(x, caesar)\}$	clause from KB
6.	$\{Loyal(x, f(x))\}$	clause from KB
7.	$\{\neg TryKill(x, y), Ruler(y)\}$	clause from KB
8.	$\{\neg TryKill(x, y), \neg Loyal(x, y)\}$	clause from KB
9.	$\{TryKill(marcus, caesar)\}$	clause from KB
10.	$\{\neg Hate(z, caesar), A(z)\}$	negated query with answer predicate
11.	$\{Loyal(marcus, caesar), Hate(marcus, caesar)\}$	resolve 2,5 with $\sigma = \{x/marcus\}$
12.	$\{\neg Loyal(marcus, caesar)\}$	resolve 8,9 with $\sigma = \{x/marcus, y/caesar\}$
13.	....	resolve ....
14.	....	resolve ....

Now, represent this resolution proof as a resolution graph!

(b) Determine whether the following sentence is valid (i.e., a tautology) using FOL Resolution:

$$\exists x \forall y \forall z((P(y) \Rightarrow Q(z)) \Rightarrow (P(x) \Rightarrow Q(x))).$$

Solution:
Please check this video that explains this solution in more detail.
Because the existential quantifier is outside the scope of both universal quantifiers, we don't need to use a Skolem function, just a Skolem constant. Here we will use the constant A, and the substitution x/A.

$\forall y \forall z[(P(y) \Rightarrow Q(z)) \Rightarrow (P(A) \Rightarrow Q(A))]$

We have an over-all implication of $(P(y) \Rightarrow Q(z)) \Rightarrow (P(A) \Rightarrow Q(A))$, where $(P(y) \Rightarrow Q(z))$ is the antecedent and $(P(A) \Rightarrow Q(A))$ is the consequent (what we're trying to prove).

Putting the antecedent into CNF gives:

$\neg P(y) \vee Q(z)$

Putting the the consequent into CNF gives:

$\neg P(A) \vee Q(A)$

Exercise 5 continues on the next page...

As we are trying to prove the consequent by refutation, we must negate it:

$P(A) \wedge \neg Q(A)$

Which gives us the clauses:

1. $\{\neg P(y), Q(z)\}$
2. $\{P(A)\}$
3. $\{\neg Q(A)\}$

   Finally, we can solve this by resolution:

4. $\{Q(z)\}$ $[1, 2]$
5. $\{\}$ $[3, 4]$

As we have derived the empty set using resolution, it must be a tautology.

(c) (From [Brachman and Levesque 2005]) Victor has been murdered, and Arthur, Bertram, and Carleton are the only suspects (meaning exactly one of them is the murderer). Arthur says that Bertram was the victim's friend, but that Carleton hated the victim. Bertram says that he was out of town the day of the murder, and besides, he didn't even know the guy. Carleton says that he saw Arthur and Bertram with the victim just before the murder. You may assume that everyone—except possibly for the murderer—is telling the truth.

You are also given the following general facts:

- anyone who was with Victor on the day of the murder, must have been in town;
- a person who is with someone, must also know them;
- a friend knows the person who they call a friend;
- a person who hates someone must also know them.

You should use the following set of predicates when expressing the general facts and statements made by the suspects:

- $I(x)$: means x is innocent;
- $F(x, y)$: means x is a friend of y;
- $H(x, y)$: means x hates y;
- $T(x)$: means x is in town;
- $K(x, y)$: means x knows y; and
- $W(x, y)$: means x is with y.

i. Use Resolution to find the murderer. In other words, formalize the facts as a set of clauses, prove that there is a murderer (here, we assume that there is only one murderer), and extract their identity from the derivation.

Solution: (<u>Note:</u> this is a partial solution)

If we assume that if someone is innocent, then they are telling the truth, we can derive the following formulas from the statements given:

i. $I(Arthur) \Rightarrow F(Bertram, Victor) \wedge H(Carleton, Victor)$
ii. $I(Bertram) \Rightarrow \neg T(Bertram) \wedge \neg K(Bertram, Victor)$
iii. $I(Carleton) \Rightarrow W(Arthur, Victor) \wedge W(Bertram, Victor)$

Converting these to CNF and putting into set form we have:

i. $\{\neg I(Arthur), F(Bertram, Victor)\}, \{\neg I(Arthur), H(Carleton, Victor)\}$
ii. $\{\neg I(Bertram), \neg T(Bertram)\}, \{\neg I(Bertram), \neg K(Bertram, Victor)\}$
iii. .....

From the four general facts we are given, we can obtain the formulas:

 Exercise 5 continues on the next page...

iv. $\forall x.W(x, Victor) \Rightarrow T(x)$
v. $\forall x \forall y.W(x, y) \Rightarrow K(x, y)$
vi. .....
vii. .....

Converting these to CNF and putting into set form we have:

iv. $\{\neg W(x, Victor), T(x)\}$
v. $\{\neg W(x, y), K(x, y)\}$
vi. $\{\neg F(x, y), K(x, y)\}$
vii. $\{\neg H(x, y), K(x, y)\}$

Finally, we assume that there is one and only one murderer, and it must be one of Arthur, Bertram or Carleton, which we can express as:

viii. $\forall x.\neg I(x) \Rightarrow x = Arthur \vee x = Bertram \vee x = Carleton$
ix. $\forall x \forall y.\neg I(x) \wedge \neg I(y) \Rightarrow x = y$
x. $\neg I(Arthur) \vee \neg I(Bertram) \vee \neg I(Carleton)$

Converting these to CNF and putting into set form we have:

viii. .....
ix. $\{I(x), I(y), x = y\}$
x. $\{\neg I(Arthur), \neg I(Bertram), \neg I(Carleton)\}$

We also need to assert that $Arthur$, $Bertram$, $Carleton$, and $Victor$ are all different people! This gives rise to the following unit clauses:

xi. $\{\neg(Arthur = Bertram)\}, \{\neg(Arthur = Carleton)\}, \{\neg(Arthur = Victor)\}, \ldots$

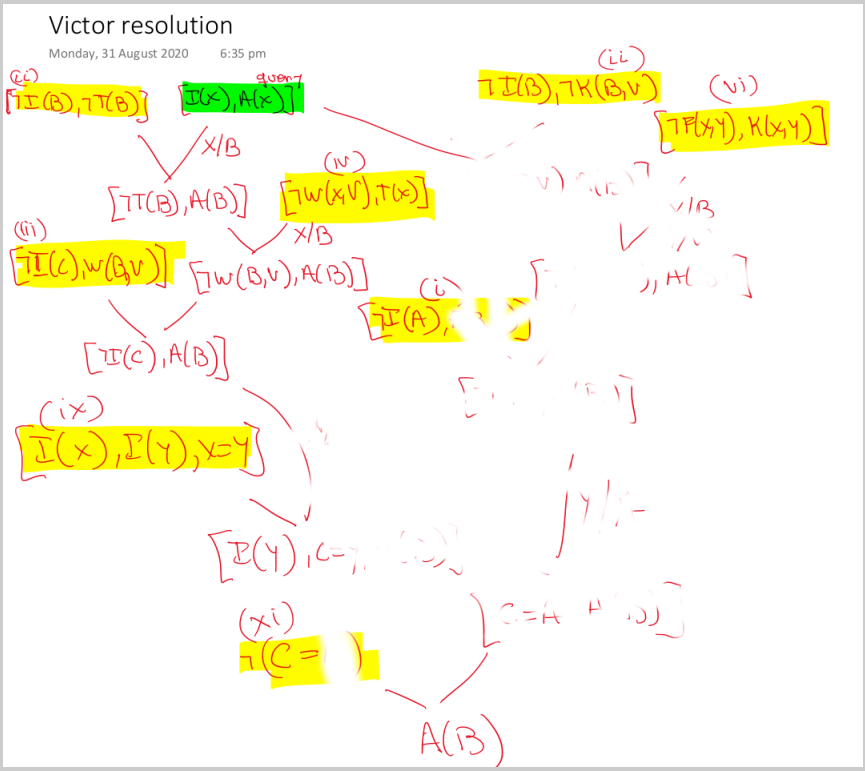
All the above formulas (i) to (xi) compromise the knowledgebase representing what is known about the case.

Finally, we want to find whether someone is guilty and who that person is (Victor is dead, after all!). So, our query is:
$\exists x.\neg I(x)$
The negated query is then $\forall x.I(x)$ (everyone is innocent!), which with the answer predicate included, yields the following clause to be considered:
$\{I(x), A(x)\}$

Remember to complete the missing . . . parts above first!

Next, let's do the resolution proof (yellow highlighted formulas come from the knowledgebase (i)-(xi); yellow highlighted represents the clause coming from the query):

 Exercise 5 continues on the next page. . .



With time, my proof has been partly faded out... *can you fill the gaps?*
One can also do a so-called **model-theoretic proof**, by analysing what would need to be true in any model of the knowledgebase, that, any interpretation that makes formulas (i) to (xi) true.

Suppose that M is a model of all formulas (i)-(xi), and suppose further that $M \models I(Bertram)$, that is $Bertram$ is innocent in interpretation M.

Then, due to (ii), we know that $M \models \neg T(Bertram)$ This together with (iv) implies that $M \models \neg W(Bertram, Victor)$. Due to (ii), we derive then that $M \models \neg I(Carleton)$.

Also due to (ii) and the fact that $M \models I(Bertram)$, we conclude that ..... This together with (vi) implies that ..... Due to (i), we conclude $M \models \neg I(Arthur)$.

So, we have $M \models \neg I(Arthur) \wedge \neg I(Carleton)$, that is, both $Arthur$ and $Carleton$ are guilty! Using formula (ix), ... But that is not the case, as we know from formula (xi) that $M \models \neg(Arthur = Carleton)$!

So, we reach a contradiction and hence it cannot be the case that ..... Thus, any model M of the knowledgeable (i)-(xi) must be such that ....., and Bertram is therefore guilty!

(Note that, using (ix) and (xi), we can derive that $M \models I(Arthur)$ and $M \models I(Carleton)$.

ii. Suppose we discover that we were wrong—we cannot assume that there was only a single murderer (there may have been a conspiracy). Show that in this case the facts do not support anyone's guilt. In other words, for each suspect, present a logical interpretation that supports all the facts, but where that suspect is innocent (and maybe the other two are guilty!).

Solution:
Here, we no-longer know that there was only one murderer: there could be more than one. So, we drop constraint formula (ix.) above.
Under the new formalization (i.e., previous formalization except formula ix.), we are to show that there is no-one who is definitively guilty. Technically, for each individual suspect, we construct an interpretation, a "possible world", that is compatible with all the information at hand, but under

which the suspect is indeed innocent.

So, let's prove that, in the new formalization, $Bertram$ can indeed be innocent. Take any interpretation M such that:

- $I(Bertram)$ is true, but $I(Arthur)$ and $I(Carleton)$ are false in M. That is, Bertram is innocent and the other two are guilty! Observe this is now possible because we don't have to satisfy formula $ix.$ anymore: a conspiracy is possible!
- $T(Bertram)$, $W(Bertram, Victor)$, $F(Bertram, Victor)$ and $H(Bertram, Victor)$ are false in M.

The truth value of the remaining predicate instances can chosen arbitrary to finish interpretation M, for example, everything else can be false in M. On can then verify that M indeed satisfies all the formulas of the formalization (except for ix. which is left out in this exercise). Thus, unlike in the previous part, we now *cannot conclude* with certainty that $Bertram$ was the murderer, as we have just showed a possible way things could turn out to be where $Bertram$ is innocent.

Now your turn to show the other two cases, that is, one interpretation under which Arthur is innocent, and one in which Carleton is innocent!

Question: could there have been a conspiracy if Arthur is innocent?

End of tutorial worksheet

COSC1127/1125 Artificial Intelligence

School of Computing Technologies
Semester 2, 2022
Prof. Sebastian Sardiña
Tutorial No. 6
Automated Planning

PART 1: Conceptual

Explain informally but clearly and precisely.

(a) Closed world assumption.

Solution: The assumption that when a proposition cannot be proven true, it is taken as false (like in a relational database: if the tuple is not in the table, it is not true).

(b) The frame problem. Provide two examples.

Solution:
The challenge of representing the effects of action in logic without having to represent explicitly a large number of intuitively obvious non-effects. See this entry in Stanford Encyclopedia of Philosophy.

(c) The three components an action using the STRIPS language? Explain, in your own words, each each of the components are meant to define and capture. Give an example of a STRIPSaction.

Solution: A STRIPS action is built from three lists: precondition, add, and delete lists.
Example here

(d) At least 2 limitations of STRIPS language to represent actions.

Solution: Cannot represent stochastic actions, or actions that have conditional effect.

(e) The differences and similarities between problem solving/search and planning.

Solution:
Both are concerned with getting from a start state to a goal using a set of defined operations or actions.
In planning, however, we use a structured language to represent states, goals, and plans, which allows for generic solver algorithms that among other things can automatically extract heuristic functions. In search, we need to hard-code the heuristic function depending on which particular problem we are solving.

(f) What is the difference between path/motion planning and automated planning.

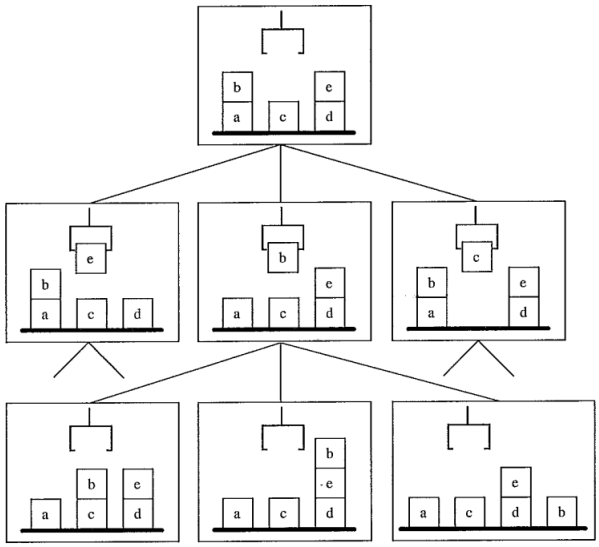
Solution: Path planning is a domain dependent task: it is known what the problem is, namely, navigating from source to destination in a network (of roads, for example). Automated planning implies not knowing what the problem is, it could be a motion planning task or playing tic-tac-toe, or a robot collecting and delivering mail in a building.

(g) Explain what GraphPlan is and what is it used for.

Solution: Graphplan is a technique to do planning by constructing a layered graph and is also used to obtain, automatically, domain-independent heuristics from a domain problem specification given in a certain factored representation (like STRIPS or PDDL).

PART 2: Blocks World

Consider the following blocks world scenario:



Suppose we use the following fluents (predicates in logic) to represent a states:

- $OnTable(x)$: block x is on the table.
- $On(x, y)$: block x is on top of block y.
- $Clear(x)$: there is nothing on top of block x.
- $Gripping(x)$: the gripper is holding block x.
- $Empty$: the gripper is not holding anything.

(a) Specify the initial state in logical representation, that is, using conjunction of atoms and relying on the close world assumption.

Solution: (Note: this is a partial solution)

The initial state of the blocks world above may be represented by the following set of predicates:

$$OnTable(a) \wedge OnTable(c) \wedge .... \wedge On(b, a) \wedge On(e, d) \wedge Clear(b) \wedge Clear(c) \wedge Clear(e) \wedge ....$$

(b) Write the STRIPS representation for operators $pickup(x)$, $putdown(x)$, $stack(x, y)$, and $unstack(x, y)$. Should a block be made clear stacking or putting it down?

Solution: (Note: this is a partial solution)

$pickup(x)$:

Precondition:	$Empty, Clear(x), OnTable(x)$
Add List:	$Gripping(x)$
Delete List:	$OnTable(x), Clear(x), Empty$

$\underline{putdown(x)}$:
Precondition: $Gripping(x)$
Add List: $OnTable(x), Empty, Clear(x)$
Delete List: $Gripping(x)$
$\underline{stack(x,y)}$:
Precondition: $Clear(y), Gripping(x)$
Add List: ....
Delete List: $Clear(y), Gripping(x)$
$\underline{unstack(x,y)}$:
Precondition: $Clear(x), Empty, On(x,y)$
Add List: ....
Delete List: ....

$Clear(x)$ implies that the block is clear to be stacked on; and one cannot stack something on top of something that is in the gripper. More importantly, unless equality constraints are used explicitly in the PDDL description, it could lead to an action that stack a block onto itself! Why? Consider a state such that $Ontable(a) \wedge Clear(a) \wedge Gripping(b) \wedge Clear(b)$.

(c) What is the new state if we apply operator-action $unstack(e, d)$ to the initial state.

Solution:

$OnTable(a) \wedge OnTable(c) \wedge OnTable(d) \wedge On(b,a) \wedge \boldsymbol{Gripping(e)} \wedge Clear(b) \wedge Clear(c) \wedge \boldsymbol{Clear(d)}$

(d) Explain in English what frame axioms would be required in this domain. Explain how those frame axioms were respected in the question before when $unstack(e, d)$ was executed.

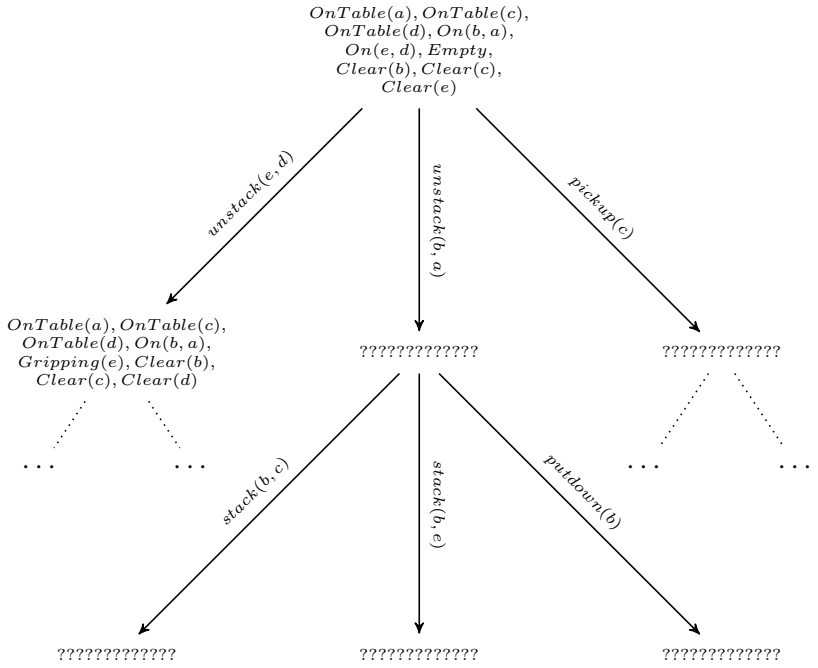
Solution: In the above, we can see as a result of applying the new operator, new predicates have been added to the new state, whereas other predicates such as $On(e, d) \wedge Empty$ have been deleted from the new state because they are no longer valid.

You will notice that predicates such as $OnTable(a) \wedge OnTable(c) \wedge OnTable(d)$ continue to remain true in the new state, thus respecting the frame axioms one would have to model the domain.

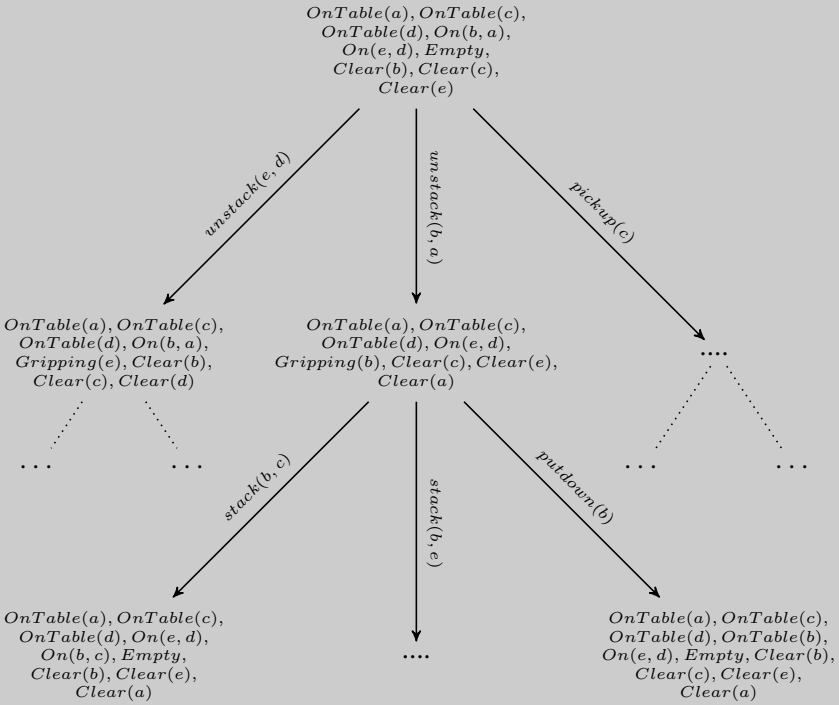
Frame axioms encode the knowledge that the things that remain the same when not affected by an action. While this comes as common sense knowledge for humans, a formal model for automatic reasoning (as computers need) require that this knowledge is encoded.

The semantics of planning languages like STRIPS and PDDL already have built-in the frame axioms: if a predicate is not affected by an operator, then its truth value remains the same.

(e) Use the operators (and the implicit frame axioms) of the previous question to complete (by filling ?????) part of the search state space of the blocks world given in the figure above. The root of the tree should be the logical initial state representation and edges should represent application of ground operators.

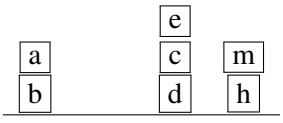


Solution: (Note: this is a partial solution)

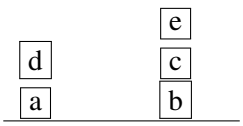


(f) Show an optimal plan that solves this problem:

Initial state:



Goal:



Solution:

There are multiple optimal solutions, here is one:

$unstack(a, b), putdown(a), unstack(e, c), putdown(e), unstack(c, d), stack(c, b),$
$pickup(e), stack(e, c), pickup(d), stack(d, a)$

If you have not come up with one yourself and just read ours, now get one by yourself.

(g) How many solutions are there for the above question? How many optimal solutions?

Solution:

There are infinitely many solutions, as you can start by stacking and unstacking the same block repeatedly before proceeding. There are only 8 optimal solutions (the first and last pairs of actions can be swapped independently, and for each of those 4 solutions, we can stack and unstack e with m instead of putting it on the table temporarily).

(h) Is the goal state correctly defined? Explain either way.

Solution: Yes, a goal is not a full state, it is a condition that any state should satisfy. Hence, in this condition it does not matter where blocks m and h end up being.

Said so, one could argue whether the goal picture specify whether blocks d and e should be clear or not. If block m ends up on top of block e, is it a goal state?

(i) What would GraphPlan give as heuristic value for the initial state? To find out, build the planning graph starting from the initial state. Is the value of the heuristic less, equal, or greater than the true length of the plan you found in the previous question? Explain why.

Solution:

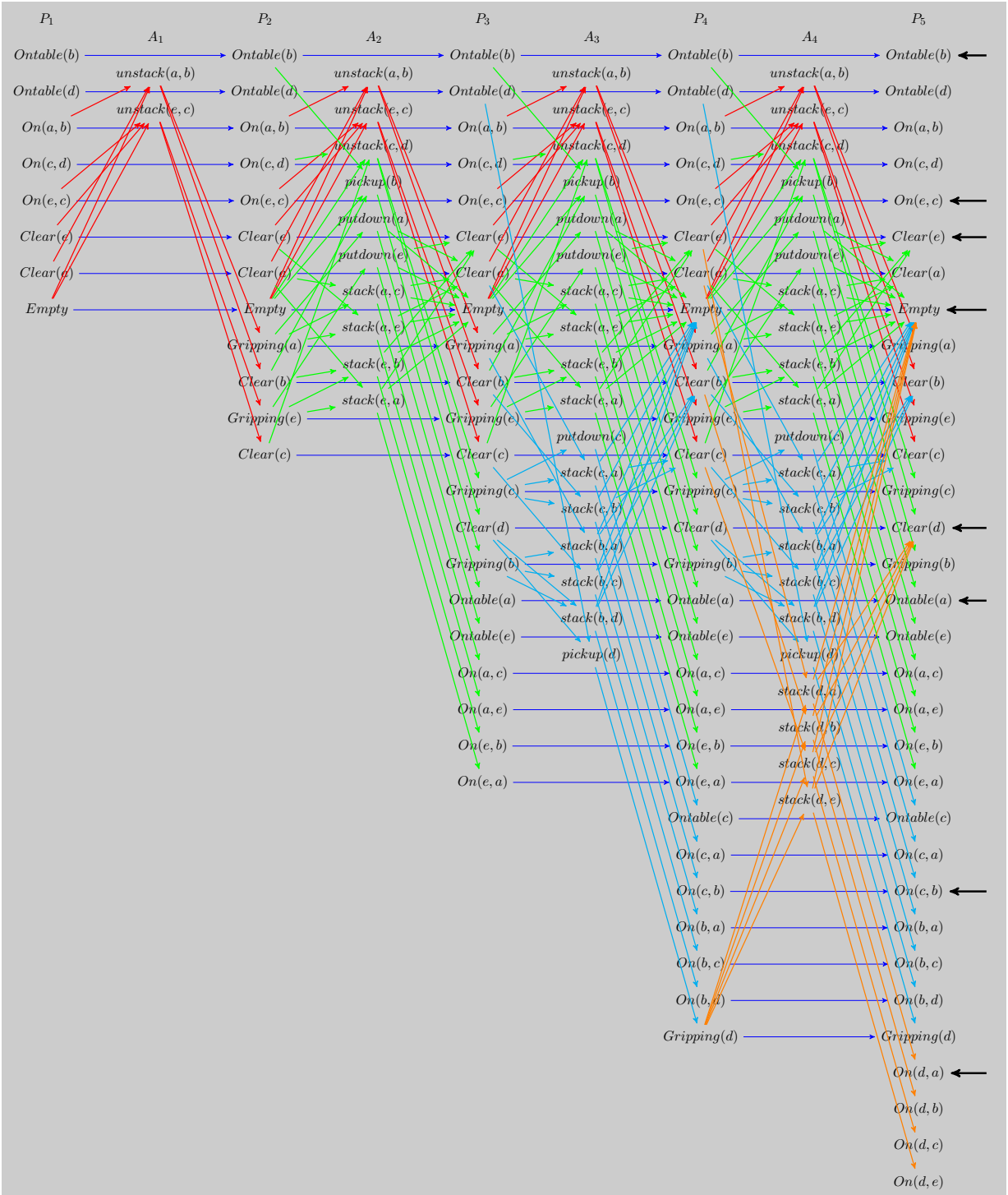
Please check this video that explains this solution in more detail.

The optimal plan has 10 actions, but GraphPlan should give you a lower heuristic value because, among other things, it will be able to hold many blocks at the same time, put them on the table all at the same time, and the original sub-tower $On(e, c)$ will always stay true!

We construct a GraphPlan graph by iteratively alternating proposition layers (P_i) and action layers (A_i). Starting with the set of initial propositions in P_1, the graph can be constructed by the following rules:

- an action is in A_i iff all of its preconditions are in P_i; and,
- a proposition is in P_{i+1} iff it is in the add list for an action in A_i (except for P_1).

Here is the graph representing the GraphPlan algorithm process for the this blocksworld problem (to save space we will ignore blocks m and h, as they are not useful in this instance):



Now, as the goal is $Ontable(a) \wedge Ontable(b) \wedge On(d, a) \wedge On(c, b) \wedge On(e, c) \wedge Clear(d) \wedge Clear(e)$, and all of those propositions are in layer P_5 (indicated with black arrows), we can say the heuristic lower bound for the plan is 4.

Some actions, such as those that undo previous actions (e.g., $unstack(x, y) \rightarrow stack(y, x)$) have been omitted to save space. You may have noticed that the number of propositions and actions monotonically increases with each layer. For very large planning problems, this can quickly blow up, beyond control (especially when doing it by hand!). Therefore, we can add an additional set of exclusion relations that are maintained throughout the process, which help to eliminate incompatible propositions and actions, hence reducing the graph size — but this is beyond the scope of this course.

(j) Suppose that each block has a new property of colour. All the rest of the domain remains the same. What would you change to your representation and corresponding STRIPS modeling?

Solution: Nothing. The new property of colour is not required for any preconditions of the operators. So it is irrelevant for the problem at hand and does not need to be taken into account.

(k) Suppose you want to represent the action of dropping an object. Its effect is that the object is on the floor and depending on whether it is fragile or not, it may or may not be broken. Can you represent such action in STRIPS representation? Either provide the representation or explain why not. Can you think of a way that you could work around this limitation by defining multiple actions?

Solution: It is not possible to represent such operator in STRIPS because it requires the ability to represent so-called *conditional effects*, that is, effects that depend on the context of where the action is applied (in our case, whether the object is fragile or not, which is represented by predicate $Fragile(x)$)

It is however possible to represent it in PDDL as PDDL supports conditional effects using the construct `(when (condition effect))` as follows:

```
(:action drop
 :parameters (?x)
 :precondition (and (Holding ?x))
 :effect (and (OnFloor(?x) (when (Fragile ?x) (Broken ?x)) )
```

If one wants to stick to STRIPS representations, the workaround is to use two actions $dropFragile(x)$ and $dropNotFragile(x)$, then we could have preconditions on the fragility (or lack thereof) of the object x being dropped, so the effect is no longer conditional.

PART 3: Time to code in PDDL!

In this question, you will code a problem in PDDL and let the automated planning system find the solution for you! To do so, you will use the planning.domain's editor, where you can code your domain and a specific problem and use an online solver to get the plan (if any exists!).

Check the video that I did here to show you how to use planning.domain's editor and its animation facility!

For PDDL, besides the book, you may want to check the PDDL quick tutorial notes here and here. Another useful material is this PDDL by Example pack of slides. Finally a nice web-based PDDL reference can be found here.

(a) First model the Blocks World domain we have seen above. Take the template file domain.pddl encoding the domain in PDDL and fill the gaps in the editor. You can just drag and drop the file and it will load it.

Remember that the domain file only contains the fluents and operators. It is meant to be usable with any specific problem instance.

Solution: Here is the start of the domain file:

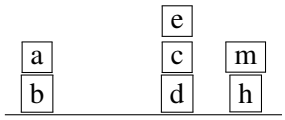
```
(define (domain BLOCKS-AI20)
(:requirements :strips)
(:predicates (on ?x ?y)
       (ontable ?x)
       (clear ?x)
       (empty)
       (gripping ?x)
       )
```

There is now very little to be filled...

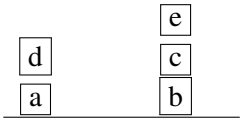
(b) Next, encode the *specific problem* we saw above by completing the following template problem.pddl file.

A problem file specifies what domain the problem is based on, the actual objects in the problem (in our case, what blocks there are), the initial state, and the condition for a state to be considered a goal state.

Initial state:



Goal:



Solution: Your turn!

(c) Find a solution by hitting the button `Solve!` in the web interface. If your domain and problem files are OK, you should get the plan with 10 actions!

Solution:

```
(unstack a b)
(putdown a)
(unstack e c)
(putdown e)
(unstack c d)
(stack c b)
(pickup e)
(stack e c)
(pickup d)
(stack d a)
```

(d) Finally, animate it using the *Planimation* plugin using the blockworld_AP.pddl animation file.

Solution: Your turn!

(e) Are you still curious? Do you want to see more involved domains with larger plans? Check the **parcprinter** domain, from a real-wrold case:

> This domain models the operation of the multi-engine printer, for which one prototype is developed at the Palo Alto Research Center (PARC). This type of printer can handle multiple print jobs simultaneously. Multiple sheets, belonging to the same job or different jobs, can be printed simultaneously using multiple Image Marking Engines (IME). Each IME can either be color, which can print both color and black&white images, or mono, which can only print black&white image. Each sheet needs to go through multiple printer components such as feeder, transporter, IME, inverter, finisher and need to arrive at the finisher in order. Thus, sheet $(n + 1)$ needs to be stacked in the same finisher with sheet n of the same job, but needs to arrive at the finisher right after sheet n (no other sheet stacked in between those two consecutive sheets). Since IMEs are heterogeneous (mixture of color and mono) and can run at different speeds, optimizing the operation of this printer for a mixture of print jobs, each of them is an arbitrary mixture of color/b&w pages that are either simplex (one-sided print) or duplex (two-sided print) is a hard problem.

Use this already prepared session to solve one problem instance. How many nodes did the planner expanded? How many did it generate but did not manage to expand? Post those stats in the forum!

End of tutorial worksheet

COSC1127/1125 Artificial Intelligence

School of Computing Technologies
Semester 2, 2022
Prof. Sebastian Sardiña

Tutorial No. 7
Probability Reasoning

We use $P(\cdot)$ (sometimes also written $\Pr(\cdot)$) to refer to a probability function or probability distribution. When using an upper letter (e.g., X or $Cavity$), we refer to the random variable; when using lowercases we refer to a specific value of the corresponding random variable. So, $P(Cavity)$ is a probability distribution over variable $Cavity$; whereas $P(cavity)$ is a shorthand for $P(Cavity = true)$ and $P(\neg cavity)$ is a shorthand for $P(Cavity = false)$.

PART 1: Probabilities

(a) Prove that if X is a random variable over a finite range R, then $P(\bigvee_{x \in R} X = x) = 1$

Solution: (Note: this is a partial solution)
This follows from the axiom $P(a \vee b) = P(a) + P(b) - P(a \wedge b)$ and that $P(X = x_1 \wedge X = x_2) = 0$ as a random variable can only take one value in an event. With that key insight, now build the full proof for the statement...

(b) Would it be rational for an agent to hold the three beliefs $P(A) = 0.4$, $P(B) = 0.3$, and $P(A \vee B) = 0.5$? If so, what range of probabilities would be rational for the agent to hold for $A \wedge B$? Make up a table like Figure 13.2 or this one, and show how it supports your argument about rationality. Then draw another version of the table where $P(A \vee B) = 0.7$. Explain why it is rational to have this probability, even though the table shows one case that is a loss and three that just break even. (Hint: what is Agent 1 committed to about the probability of each of the four cases, especially the case that is a loss?)

Solution: (Note: this is a partial solution)
Probably the easiest way to keep track of what's going on is to look at the probabilities of the atomic events. A probability assignment to a set of propositions is consistent with the axioms of probability if the probabilities are consistent with an assignment to the atomic events that sums to 1 and has all probabilities between 0 and 1 inclusive. We call the probabilities of the atomic events a, b, c, and d, as follows:

	B	$\neg B$
A	a	b
$\neg A$	c	d

We then have the following equations:

$$P(A) = ... = 0.4$$
$$P(B) = ... = 0.3$$
$$P(A \vee B) = ... = 0.5$$
$$P(True) = a + b + c + d = 1$$

From these, it is straightforward to infer that $a = 0.2$, $b = 0.2$, $c = 0.1$, and $d = 0.5$. Therefore, $P(A \wedge B) = a = 0.2$. Thus the probabilities given are consistent with a rational assignment, and the probability $P(A \wedge B)$ is exactly determined.
If $P(A \vee B) = 0.7$, then $P(A \wedge B) = a = 0$. Thus, even though the bet outlined in Figure 13.3 loses if A and B are both true, the agent believes this to be impossible so the bet is still rational.

	toothache		*¬toothache*	
	catch	*¬catch*	*catch*	*¬catch*
cavity	0.108	0.012	0.072	0.008
¬cavity	0.016	0.064	0.144	0.576

Figure 13.3 A full joint distribution for the *Toothache*, *Cavity*, *Catch* world.

(c) Consider the domain of dealing 5-card poker hands from a standard deck of 52 cards, under the assumption that the dealer is fair.

i. How many atomic events are there in the joint probability distribution (i.e., how many 5-card hands are there)?

Solution: (Note: this is a partial solution)

This is a classic combinatorics question. The point here is to refer to the relevant axioms of probability, principally, the following axioms:

(1) All probabilities are between 0 and 1. $0 \leq P(A) \leq 1$
(2) $P(True) = 1$ and $P(False) = 0$
(3) The probability of a disjunction is given by $P(A \vee B) = P(A) + P(B) - P(A \wedge B)$

The question also helps students to grasp the concept of joint probability distribution over all possible states of the world.

It is important to note here that the hand $\{\clubsuit 2, \diamondsuit 3, \heartsuit 4, \diamondsuit 5, \spadesuit 6\}$, is identical to the hand $\{\spadesuit 6, \diamondsuit 5, \heartsuit 4, \diamondsuit 3, \clubsuit 2\}$. If the order of the cards dealt mattered, then the number of hands would simply be $\frac{52!}{47!}$, or $52 \times 51 \times 50 \times 49 \times 48$, because there are 52 choices for the first card, 51 for the second, 50 for the third, and so on.
Generally though, when dealing a hand of cards, it doesn't matter the order in which they are dealt. So, you need to divide this number by the number of possible permutations for a hand of 5 cards, which is $5 \times 4 \times 3 \times 2 \times 1 = 5!$. This means that the total number of 5 card hands that are possible in a 52 card deck is .... .
In combinatorics this is written as the binomial coefficient $\binom{52}{5}$, and means: "out of 52 cards, choose 5". In general, $\binom{n}{k} = \frac{n!}{k!(n-k)!}$.

ii. What is the probability of each atomic event?

Solution: (Note: this is a partial solution)

By the fair-dealing assumption, each of these is equally likely. Each hand therefore occurs with probability . . . (your turn)

iii. What is the probability of being dealt a royal straight flush (the ace, king, queen, jack and ten of the same suit)?

Solution: (Note: this is a partial solution)

There are four hands that are royal straight flushes (one in each suit). By axiom 3, since the events

are mutually exclusive, the probability of a royal straight flush is just the sum of the probabilities of the atomic events, i.e., . . . (your turn)

iv. What is the probability of being dealt four-of-a-kind (i.e., four cards of different suit but same face value)?

Solution: (Note: this is a partial solution)

Again, we examine the atomic events that are four-of-a-kind events. There are 13 possible kinds and for each, the fifth card can be one of 48 possible other cards. The total probability is therefore . . . (your turn)

PART 2: Conditional probability

(a) Prove, formally, that $P(A \mid B \wedge A) = 1$.

Solution: (Note: this is a partial solution)

You need to use the definition of conditional probability, $P(X \mid Y) = P(X \wedge Y) \mid P(Y)$, and the definitions of the logical connectives. It is not enough to say that if $B \wedge A$ is "given", then A must be true. From the definition of conditional probability and the fact that $A \wedge A \iff A$ and that conjunction is commutative and associative we have,

$$P(A \mid B \wedge A) = ... = 1$$

(b) Consider again the domain of dealing 5-card poker hands from a standard deck of 52 cards, under the assumption that the dealer is fair. You are told that the probability drawing two cards from a deck of 52 and them both being an ace is $\frac{1}{221}$. You take the deck and draw the first card — it is the ace of spades! What is the probability that the second card you draw will also be an ace? (even though this is easy to work out using binomials, use conditional probability for this question)

Solution: (Note: this is a partial solution)

Even though we are given the suit of the first ace, that information is immaterial as the probability you are given does not specify suit, so we can ignore it. If we let $P(A)$ be the probability of drawing the first ace, its value is simply $\frac{4}{52}$. Let $P(A \wedge B)$ be the probability of drawing two aces in a row, which is $\frac{1}{221}$. We can work out the probability of drawing a second ace, *given* we have already drawn an ace, $P(B \mid A)$, using the definition of conditional probability.

$$P(B \mid A) = ... = \frac{\frac{1}{221}}{\frac{4}{52}} = \frac{52}{4 \times 221} = \frac{3}{51}$$

(c) Given the full joint distribution shown in the table below, calculate the following:

	$toothache$		$\neg toothache$	
	$catch$	$\neg catch$	$catch$	$\neg catch$
$cavity$	.108	.012	.072	.008
$\neg cavity$	.016	.064	.144	.576

Note that $P(Cavity)$ denotes a vector of values for the probabilities of each individual state of Cavity. Also note here we use the uppercase (e.g., $Cavity$) to denote a variable, whereas lowercase (e.g., $cavity$) to denote a constant.

i. $P(toothache)$

Exercise 2 continues on the next page. . .

Solution: (Note: this is a partial solution)

We need to sum up over all entries in the table that relate to $toothache$.
$P(toothache) = \ldots$ (your turn)

ii. $P(Cavity)$

Solution: (Note: this is a partial solution)

This time we are dealing with a variable, so we need to give a vector of values, i.e., $P(A) = \langle P(a), P(\neg a)\rangle$.
$P(Cavity) = \ldots$ (your turn)

iii. $P(Toothache \mid cavity)$

Solution: (Note: this is a partial solution)

Here we have a conditional probability of a variable, given a constant. So the solution will be a vector again of the form $P(A \mid b) = \left\langle \frac{P(a \wedge b)}{P(b)}, \frac{P(\neg a \wedge b)}{P(b)} \right\rangle$
$P(Toothache \mid cavity) = \ldots$ (your turn)

iv. $P(Cavity \mid toothache \vee catch)$

Solution: (Note: this is a partial solution)
$P(Cavity \mid toothache \vee catch) = \ldots$ (your turn - remember that the values in your vector should add up to 1)

(d) After your yearly checkup, the doctor has bad news and good news. The bad news is that you tested positive for serious disease and that the test is 99% accurate (i.e., the probability of testing positive when you do have the disease is 0.99, as is the probability of testing negative when you don't have the disease). The good news is that this is a rare disease, striking only 1 in 10,000 people of your age. Why is it good news that the disease is rare? What are the chances that you actually have the disease?

Solution: (Note: this is a partial solution)

We are given the following information:

$$P(test \mid disease) = \ldots\ldots\ldots\ldots$$
$$P(\neg test \mid \neg disease) = \ldots\ldots\ldots\ldots$$
$$P(disease) = \ldots\ldots\ldots\ldots$$
$$test$$

where $test$ means that the test is positive. What the patient is concerned about is $P(disease \mid test)$. Roughly speaking, the reason it is a good thing that the disease is rare is that $P(disease \mid test)$ is proportional to $P(disease)$, so a lower prior probability for $Disease$ will mean a lower value for $P(disease \mid test)$. By and large, if 10,000 people take the test, we expect 1 to actually have the disease, and most likely test positive, while the rest do not have the disease, but 1% of them (about 100 people) will test positive anyway, so $P(disease \mid test)$ will be about 1 in 100. More precisely, using the following:

 Exercise 2 continues on the next page...

$$P(disease \mid test) = \frac{P(test \mid disease)P(disease)}{P(test)}$$
$$= \ldots\ldots\ldots\ldots$$
$$= \ldots\ldots\ldots\ldots$$
$$= 0.009804$$

Note that in the above, to calculate $P(test)$, we need to sum over all other hidden variables, but in this case there is only one, that is $Disease$:

$$P(test) = P(test \wedge disease) + P(test \wedge \neg disease)$$

by the product rule, we have:

$$P(test \wedge disease) = P(test \mid disease)P(disease)$$
$$P(test \wedge \neg disease) = P(test \mid \neg disease)P(\neg disease)$$

The moral is that when the disease is much rarer than the test accuracy, a positive result does not mean the disease is likely. A false positive reading remains much more likely.
See a video explaining the solution HERE.

(e) Prove, formally, that $P(A \wedge B \wedge C) = P(A \mid B \wedge C) \times P(B \mid C) \times P(C)$.

Solution: (Note: this is a partial solution)

Starting with the definition of conditional probability,

$$P(X \mid Y) = \frac{P(X \wedge Y)}{P(Y)}$$

we can rearrange to give the product rule:

$$P(X \wedge Y) = ...$$

Using the product rule and substituting X/A and $Y/[B \wedge C]$ into $P(A \wedge B \wedge C)$, we have,

$$P(A \wedge B \wedge C) = \ldots \text{ (your turn)}$$

Finally, applying the product rule again to $P(B \wedge C)$ and substituting X/B and Y/C gives:

$$P(A \wedge B \wedge C) = \ldots \text{ (your turn)}$$

(f) Prove Bayes' Theorem: $P(A \mid B) = \frac{P(B \mid A) \times P(A)}{P(B)}$.

Solution: (Note: this is a partial solution)

The probability of two events A and B happening, $P(A \wedge B)$, is the probablity of A, $P(A)$, times the probability of B given that B has occurred, $P(B \mid A)$.

$$P(A \wedge B) = \ldots \text{ (your turn)}$$

Exercise 2 continues on the next page...

On the other hand, the probability of A and B is also equal to the probability of B times the probability of A given B.

$$P(A \wedge B) = \ldots \text{ (your turn)}$$

Equating the two yields:

$$P(B) \times P(A \mid B) = \ldots \text{ (your turn)}$$

and thus,

$$P(A \mid B) = P(A) \times \frac{P(B \mid A)}{P(B)}$$

(g) For each of the following statements, either prove it is true or give a counterexample:

i. If $P(a \mid b, c) = P(b \mid a, c)$, then $P(a \mid c) = P(b \mid c)$

Solution: True. By the product rule we know $P(b, c)P(a \mid b, c) = P(a, c)P(b \mid a, c)$, which by assumption reduces to $P(b, c) = P(a, c)$. Dividing through by $P(c)$ gives the result.

ii. If $P(a \mid b, c) = P(a)$, then $P(b \mid c) = P(b)$

Solution: (Note: this is a partial solution)
False. The statement $P(a \mid b, c) = P(a)$ merely states that a is independent of b and c, it makes no claim regarding the dependence of b and c.
A counter-example: .....

iii. If $P(a \mid b) = P(a)$, then $P(a \mid b, c) = P(a \mid c)$

Solution: (Note: this is a partial solution)
False. While the statement $P(a \mid b) = P(a)$ implies that a is independent of b, it does not imply that a is conditionally independent of b given c.
A counter-example: .....

End of tutorial worksheet

COSC1127/1125 Artificial Intelligence

School of Computing Technologies
Semester 2, 2022
Prof. Sebastian Sardiña

Tutorial No. 8
Markov Decision Processes (MDPs)

You can check this video on max/min vs arg max/arg min.

Question 1: MDPs

Recall a Markov Decision Process (MDP) represents a scenario where actions are non-deterministic. Consider the following minor gridworld example:

	1	2	3
1	20		5
2			
3	-20		

Cells are indexed by (row, column). Cells $(1,1)$, $(3,1)$ and $(1,3)$ are terminal cells, and have rewards of 20, -20 and 5, respectively. For all other cells, there is an reward of -1.[1] The agent starts at cell $(3,3)$. The agent can move in north, east, south and west directions or stay where it is. The actions are stochastic and have the following outcomes:

- If go north, 70% of the time the agent will go north, 30% will go west.
- If go east, 60% of the time the agent will go east, 40% of the time will go south.
- If go south, 80% of the time the agent will go south, 20% of the time will go east.
- If go west, 70% of the time the agent will go west, 30% of the time will go north.
- If stay in current cell, 100% of the time will stay.

If an action causes the agent to travel into a wall, the agent will stay in their current cell.

(a) Construct a MDP for this instance of gridworld. Recall a MDP consists of S (set of states), s_0 (starting state), possible terminal states, A (set of actions), T (transition model/function) and R (reward function), so all of these have to be specified. To help, consider constructing the MDP in steps:

i. What are the states in this gridworld?

Solution:
Each coordinate in the grid corresponds to a state.

[1] Note this is taking a framework where rewards are given to states (i.e., $R(s)$) and not as transitions (i.e., $R(s,a,s')$) as done in the book. As we have discussed in class, both are standard conventions and they can be reduced to each other without loss of generality.

ii. What are the starting and terminal states?

Solution: (Note: this is a partial solution)

Starting state is $(3, 3)$, terminal states are $(1, 1)$, $(3, 1)$ and .....

iii. What are the set of actions?

Solution:
For each state, actions are $\{north, east, south, west, stay\}$

iv. With cell $(3, 3)$ as the current state s, construct the transition model/function for all actions and subsequent states, i.e., $T(s, a, s')$, where a are all possible actions for $(3, 3)$ and s' are all possible successor states from $(3, 3)$.
Recall, the definition of the transition function T is,

$$T : S \times A \times S \to \mathbb{R}, (s, a, s') \mapsto P(s'|a \wedge s)$$

Solution: (Note: this is a partial solution)

- $T((3, 3), north, (2, 3)) = 0.7$ (70%, go north)
- $T((3, 3), north, (3, 2)) = 0.3$ (30%, go west)
- $T((3, 3), east, (3, 3)) = 1$ (cannot go east or south from $(3, 3)$)
- $T((3, 3), south, (3, 3)) = 1$ (cannot go east or south from $(3, 3)$)
- $T((3, 3), west, (3, 2)) =$ ..... (....., go west)
- $T((3, 3), west, (2, 3)) = 0.3$ (30%, go north)
- $T((3, 3), stay, (3, 3)) = 1.0$ (100%, stay)

v. What is the reward function?

Solution:
- $R((1, 1)) = +20$
- $R((3, 1)) = -20$
- $R((1, 3)) = +5$
- $R(\text{other states}) = -1$

(b) Using the MDP built, consider the following sequences of states that our agent progressed through. For each sequence, compute the *additive reward* and *discounted reward* (with a decay factor γ of 0.5). Take note of the differences between the two approaches.
(a) $(3, 3)$, $(2, 3)$, $(2, 2)$, $(2, 1)$, $(1, 1)$
(b) $(3, 3)$, $(3, 2)$, $(2, 2)$, $(2, 3)$, ... (repeat)

Solution: (Note: this is a partial solution)

(a) Additive: $(-1) + (-1) + (-1) + (-1) + (+20) = 16$
Discount: $(-1) + 0.5(-1) + 0.5^2(-1) + 0.5^3(-1) + 0.5^4(+20) = -0.625$ $(10/16)$

(b) Additive: $(-1) + (-1) + (-1) + (-1) + \ldots = -\infty$
Discount: ..... $= \sum_{t=0}^{\infty} 0.5^t \times (-1) = \frac{-1}{1-0.5} = -2$

Here, we assume the cumulative reward for a set of states $s_0, s_1, \ldots, s_t$ is $\sum_{i=0}^{t} R(s_i)$ (with discounting if you like). Note that another convention, as UCB slides or the textbook, is to use a reward function $R(s, a, s')$ on transitions, i.e., the reward given as you move from state s to state s' taking action a. In this scenario, you cannot have a reward for s_0, as we don't have any state it has come from previously.

Question 2: Value iteration

(a) Consider the MDP from Question 1, with a decay factor $\gamma = 1$.

i. Using value iteration, compute the utilities for each state after two iterations. For this question take the value iteration equation:

$$U_{i+1}(s) = R(s) + \gamma \left(\max_a \sum_{s' \in S} T(s, a, s') U_i(s') \right).$$

Initially, we set $U_0(s) = 0$ for all states $s \in S$. Remember terminal states do not get more rewards once reached.

Solution: (Note: this is a partial solution)

	1	2	3
1	20	12.7	5
2	12.7	−2	2.2
3	−20	−2	−2

Here is the detailed working for cell $(1, 2)$. Note that, in this case, it is clear that "moving west" is the *best action* (i.e., maximizes the second term in formula above), but in general one would need to calculate the term for every action and then keep the the max one (that is the best action to do!). Compute utility/value $U(1, 2)$ (row 1, column 2):

$$U_0(s) = 0, \text{for all sates } s \in S$$

$$U_{i+1}(s) = R(s) + \gamma \left(\max_a \sum_{s' \in S} T(s, a, s') U_i(s') \right)$$

$$U_1((1,2)) = -1 + \gamma[T((1,2), \text{west}, (1,1)) \times U_0(1,1) + T((1,2), \text{west}, (1,2)) \times U_0(1,2)]$$

$$U_1((1,2)) = -1 + 1 \times [0.7 \times 0 + 0.3 \times 0] = -1$$

(ordinarily we would compute all U_1 values for every state,
but to compute for $(1, 2)$ we can see the only other one we need is $(1, 1)$.)

$$U_1((1,1)) = 20 + \gamma[T((1,1), \text{stay}, (1,1)) \times U_0(1,1)] \qquad \text{(terminal state so must "stay")}$$

$$U_1((1,1)) = \ldots\ldots\ldots\ldots = 20$$

$$U_2((1,2)) = -1 + \gamma[T((1,2), \text{west}, (1,1)) \times U_1(1,1) + T((1,2), \text{west}, (1,2)) \times U_1(1,2)]$$

$$U_2((1,2)) = \ldots\ldots\ldots\ldots = 12.7$$

$$U_2((1,1)) = 20 \qquad \text{(terminal state so cannot take further action)}$$

ii. Repeat and determine the values after three iterations.

Solution: (Note: this is a partial solution)

	1	2	3
1	20	16.81	5
2	16.81	11.7	1.9
3	−20	−3	−0.059

Exercise 2 continues on the next page...

Continuing on from the previous question:

$$U_3((1,2)) = \ldots\ldots\ldots\ldots$$
$$U_3((1,2)) = -1 + 1 \times [0.7 \times 20 + 0.3 \times 12.7] = 16.81$$

(b) Consider the utilities after two iterations of value iteration from question a (we typically only perform this when values have converged, but for this question, do this after 2 iterations). Using the Maximum Expected Utility principle, find the best action (i.e., policy) for each state.

Recall, the equation to extract the optimal policy for a state s is:

$$\pi^*(s) = \underset{a}{\text{argmax}} \sum_{s' \in S} T(s, a, s')U(s')$$

Solution:

For the cell $(1, 2)$ we can derive that the best action to do there after 2 iterations is west, that is, $\pi((1, 2)) = \text{west}$.

(c) Consider the following policy for the MDP of question 1, with a decay factor γ of 1:

- $\pi((2, 1)) = \text{west}$.
- $\pi((2, 2)) = \text{west}$.
- $\pi((2, 3)) = \text{north}$.
- $\pi((3, 2)) = \text{north}$.
- $\pi((3, 3)) = \text{east}$.
- $\pi((1, 2)) = \text{south}$.

Evaluate this policy, i.e., compute the utility for each state, for one and two iterations (we typically can solve it as a system of simultaneous equations but for this question we use an iterative approach).

Solution:

Here we are given the policy (it is fixed), so the task is to calculate its value at every state, that is, $V_\pi(s)$ or $U_\pi(s)$. We can do that as follows:

$$U^\pi_{i+1}(s) = R(s) + \gamma \sum_{s' \in S} T(s, \pi(s), s')U^\pi_i(s').$$

Observe we do not take the $\max$ of actions because we know, at every state s, what we should do, namely, $\pi(s)$!

	1	2	3
1	20	-0.8	5
2	4.3	-2	2.2
3	-20	-7.7	-2

You can check THIS VIDEO on the solving of this problem.

(d) For any possible gridworld MDP, is the stay action ever part of an optimal policy? If yes, design a gridworld that demonstrates it, or if no prove that it is impossible. Hint: consider the reward function for non-terminal cells.

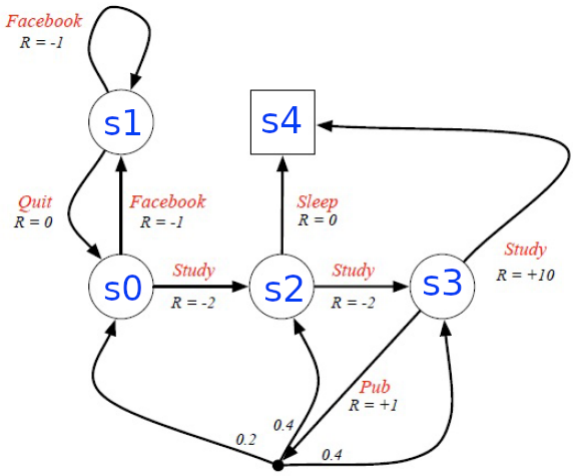
Solution: If the reward function is positive valued and large in magnitude for non terminating states, and γ is small, then it is possible for an agent to choose to stay in a state.

Question 3: More on MDP modeling

When defining an MDP, one can choose to represent the rewards per state, via a function $R(s)$ (i.e., "the immediate reward received at state s") or assign rewards to each transition via a function $R(s, a, s')$ (i.e., "the immediate reward obtaining when executing action a in state s and the system evolving to the next state s'").

The R&N book uses $R(s)$, but the more standard literature on Machine Learning uses $R(s, a, s')$. However, one can covert one to the other, possibly by adding additional states.

(a) Consider the following MDP representing the life of a student from David Silver's class (check his slides):



Can you obtain the corresponding an alternative MDP that achieves the same modeling but with rewards per state, that is, with a reward function $R(s)$?

Solution: (Note: this is a partial solution)

For each state s' in the original MDP:

- If all the transitions arriving to s' have the same reward, then just keep s' as is in the new MDP and assign that same reward to state s'.
- Otherwise, we will create a version of s' for each transition arriving to it. Concretely, for every transition $T(s, a, s')$ with reward $R(s, a, s')$, make a duplicate of state s', which we will call $s'_{s,a}$, and assign $R(s'_{s,a}) = R(s, a, s')$. Since $s'_{s,a}$ is a version of the original s', all outgoing transitions from s' need to be implemented in s' as well.

Now apply that process to the MDP above to obtain another MDP that has rewards associated to states.

State $s1$ will not need to be "copied" but the others may.

You should process one state per time. For example, $s0$ will have two versions: $s0_{s1,quit}$ and $s0_{s3,pub}$ with rewards 0 and 1, respectively. Both new states should have transitions, with probability 1 to state $s1$ under action $Facebook$. Now you do the rest... :-)

End of tutorial worksheet

COSC1127/1125 Artificial Intelligence

School of Computing Technologies
Semester 2, 2022
Prof. Sebastian Sardiña
Tutorial No. 09
Bayesian Networks

PART 1: Conditional probabilty graphs

We can use the full joint probability tables to answer any question we may have about a set of random variables; however, as the number of variables increases, the size of the joint probability table increases exponentially. Thankfully, we can use the independence and conditional independence relationships between variables to design a compact representation that greatly reduces the number of probabilities needed to define the full joint probability distribution. These structures are called *Bayesian networks* and are typically represented as directed, acyclic graphs $G = (V, E)$ where V is the set of random variables and $E \subseteq V \times V$ is the set of directed edges. Random variables are denoted by capital letters (e.g., X) and lower-case denotes the truth assignment of a variable (e.g., x or $\neg x$). Edge $(Y, X) \in E$ indicates that Y is the *parent* of X and we can denote the set of parents of X as $parents(X)$. The parents of X are the only variables that *directly* affect the probability of X. This means that given $parents(X)$, there is no other information that is useful to estimate $P(X)$, formally: $P(X \mid parents(X)) = P(X \mid V_1, V_2, \ldots, V_n)$ for all random variables $V_i \in V$.

In order to compute the full joint distribution using a Bayesian network, we need to find the product of all the conditional probabilities. We can do this using the following equation:

$$P(x_1, x_2, \ldots, x_n) = \prod_{i=1}^{n} P(x_i \mid parents(X_i)).$$

It is clear that if X_i has no parents, its "conditional" probability is simply $P(X_i)$.

Where we are making a query about a variable but we don't have full knowledge about its parents, we must sum over the hidden variables. This can be done in the following manner:

$$P(X) = \sum_{\mathbf{y}} P(X, \mathbf{y}),$$

This technique is known as *marginalisation* and here, $\mathbf{Y}$ is the vector of all hidden variables we wish to sum over. For example, in the case where there is one hidden variable Y, we have,

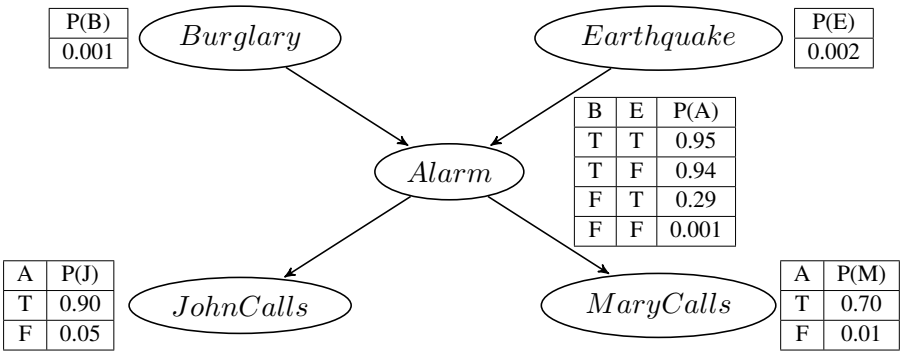
$$P(X) = P(X, y) + P(X, \neg y);$$

in the case of two hidden variables Y_1 and Y_2 we have,

$$P(X) = P(X, y_1, y_2) + P(X, y_1, \neg y_2) + P(X, \neg y_1, y_2) + P(X, \neg y_1, \neg y_2);$$

and so on. In general, we will have the sum of 2^n terms, where n is the number of hidden variables being summed over. Each of these terms will represent one of the possible truth assignments for all of the hidden variables.

Example: Consider the following Bayesian network:



(a) Give an expression for the probability $P(j, m, a, \neg e, \neg b)$.
Solution: Here, we need to find conditional probabilities for each truth assignment of the variables, based on the structure of the graphical model given. So, j depends on a; m depends on a; a depends on both e and b; and, b and e don't depend on any other variables. The total probability for the entry in the full joint table represented by $P(j, m, a, \neg e, \neg b)$ is the product of all of these conditional probabilities.
Therefore an expression for the above probability would be:
$P(j \mid a) \times P(m \mid a) \times P(a \mid \neg e, \neg b) \times P(\neg e) \times P(\neg b)$

(b) Compute the probability $P(j, m, a, \neg e, \neg b)$.
Solution: Having worked out an expression for the probability in the previous question, computing the value is a simple matter of plugging in the correct values - remembering that $P(\neg a) = 1 - P(a)$.
Therefore, the value for the above probability is:

$$\begin{aligned} P(j, m, a, \neg e, \neg b) &= P(j \mid a) \times P(m \mid a) \times P(a \mid \neg e, \neg b) \times P(\neg e) \times P(\neg b) \\ &= P(j \mid a) \times P(m \mid a) \times P(a \mid \neg e, \neg b) \times (1 - P(e)) \times (1 - P(b)) \\ &= 0.90 \times 0.70 \times 0.001 \times (1 - 0.002) \times (1 - 0.001) \\ &= 0.90 \times 0.70 \times 0.001 \times 0.998 \times 0.999 \\ &= 0.0006281113 \end{aligned}$$

(c) Give an expression for the probability $P(j, \neg m, \neg a, b)$.
Solution: Here, we are missing one of the truth assignment to the variable e, so we must marginalise over that variable by summing up all entries in the full joint table corresponding to all of the other truth assignments and when e is both true *and* when it is false.
Therefore, an expression for the above probability is:

$$\begin{aligned} P(j, \neg m, \neg a, b) &= \sum_e P(j, \neg m, \neg a, b, e) \\ &= \sum_e P(j \mid \neg a) \times P(\neg m \mid \neg a) \times P(\neg a \mid b, e) \times P(b) \times P(e) \\ &= P(j \mid \neg a) \times P(\neg m \mid \neg a) \times P(\neg a \mid b, e) \times P(b) \times P(e) \\ &\quad + P(j \mid \neg a) \times P(\neg m \mid \neg a) \times P(\neg a \mid b, \neg e) \times P(b) \times P(\neg e) \end{aligned}$$

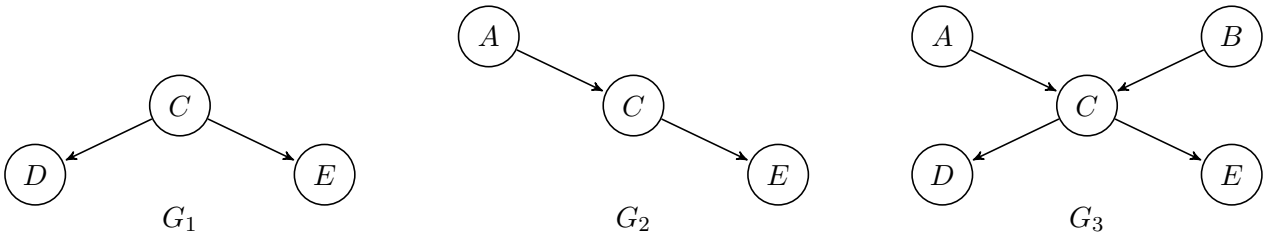
(d) Compute the probability $P(j, \neg m, \neg a, b)$.

Solution: Just plug the values in as you did before:

$$\begin{aligned}
P(j, \neg m, \neg a, b) &= P(j \mid \neg a) \times (1 - P(m \mid \neg a)) \times (1 - P(a \mid b, e)) \times P(b) \times P(e) \\
&\quad + P(j \mid \neg a) \times (1 - P(m \mid \neg a)) \times (1 - P(a \mid b, \neg e)) \times P(b) \times (1 - P(e)) \\
&= 0.05 \times 0.99 \times 0.05 \times 0.001 \times 0.002 \\
&\quad + 0.05 \times 0.99 \times 0.06 \times 0.001 \times 0.998 \\
&= 0.00000296901
\end{aligned}$$

Questions:

Consider the following directed graphical models:



Give expressions for the following probabilities using only terms of the form $P(X \mid parents(X))$:

(a) with respect to graph G_1;

i. $P(c, d, e)$

Solution:
$P(d \mid c) \times P(e \mid c) \times P(c)$

ii. $P(d, e \mid c)$

Solution: Your turn!

(b) with respect to graph G_2;

i. $P(a, c, e)$

Solution: Your turn!

ii. $P(a, e \mid c)$

Solution:

$$\begin{aligned}
P(a, e \mid c) &= \frac{P(a, e, c)}{P(c)} \\
&= \frac{P(a) \times P(c \mid a) \times P(e \mid c)}{P(c)}
\end{aligned}$$

Here, we have only the conditional probability table for C, which depends on A, so we must marginalise over A to obtain $P(c)$.

$$\frac{P(a) \times P(c \mid a) \times P(e \mid c)}{P(c)} = \frac{P(a) \times P(c \mid a) \times P(e \mid c)}{\sum_a P(c \mid a) \times P(a)} = \frac{P(a) \times P(c \mid a) \times P(e \mid c)}{P(c \mid a) \times P(a) + ...}$$

(c) with respect to graph G_3.

i. $P(a, \neg b, c, d, \neg e)$

Solution: Your turn!

ii. $P(a \mid \neg d, e)$

Solution:
Using conditional probability rule:

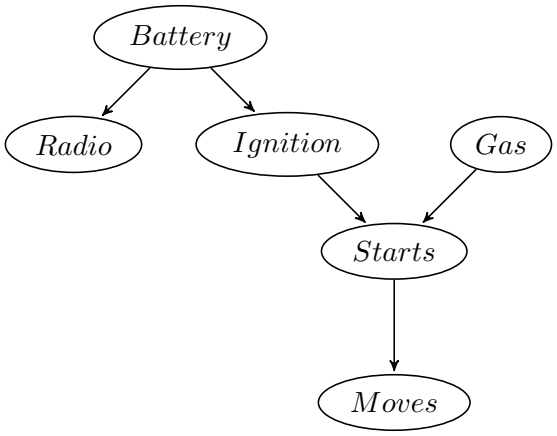
$$P(a \mid \neg d, e) = \frac{P(a, \neg d, e)}{P(\neg d, e)} = \alpha P(a, \neg d, e),$$

where α is the normalisation constant $\frac{1}{P(\neg d, e)}$.
Now, because d and e are conditional on c and c is conditional on b, we have to sum up over the hidden variables b and c.
So we have,

$$\begin{aligned}
P(a \mid \neg d, e) &= \alpha P(a, \neg d, e) \\
&= \alpha \sum_{b,c} P(a, \neg d, e, b, c) \\
&= \alpha \sum_{b,c} P(a) \times P(b) \times P(c \mid a, b) \times P(\neg d \mid c) \times .... \\
&= \alpha [P(a) \times P(b) \times P(c \mid a, b) \times P(\neg d \mid c) \times P(e \mid c) \\
&\quad + P(a) \times P(b) \times P(\neg c \mid a, b) \times P(\neg d \mid \neg c) \times P(e \mid \neg c) \\
&\quad + ... \\
&\quad + P(a) \times P(\neg b) \times P(\neg c \mid a, \neg b) \times P(\neg d \mid \neg c) \times P(e \mid \neg c)]
\end{aligned}$$

PART 2: Car diagnosis

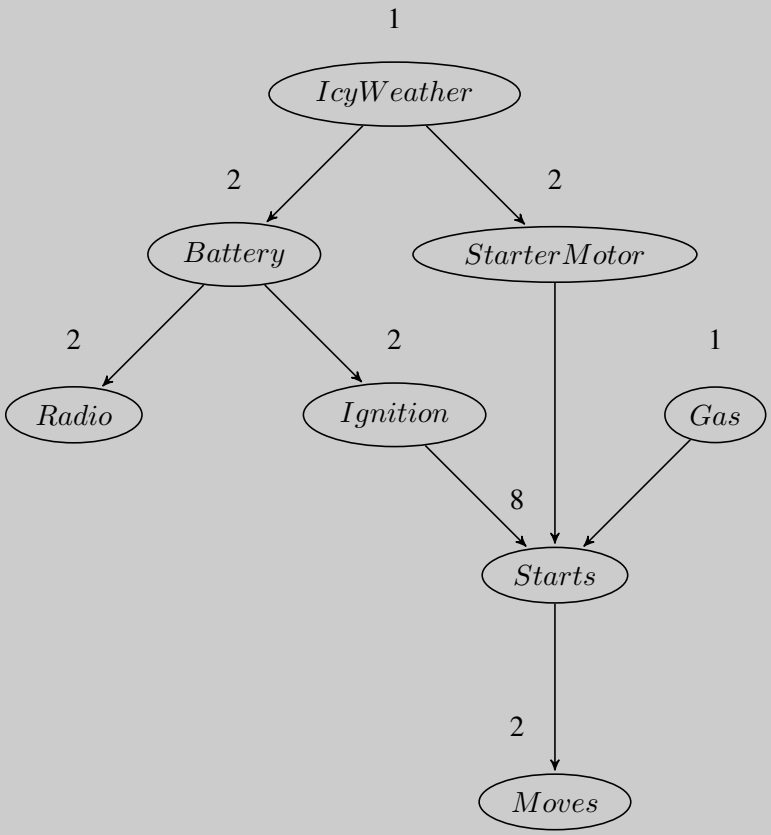
Consider the network for car diagnosis shown in the figure below:



(a) Extend the network with the Boolean variables *IcyWeather* and *StarterMotor*, by reasoning how components might affect each other. Being a design/modelling process, there may be many correct answers, just state your assumptions.

Solution:
Adding variables to an existing net can be done in two ways. Formally speaking, one should insert the variable ordering and rerun the network construction process from the point where the first new variable appears. Informally speaking, one never really builds a network by a strict ordering. Instead, one asks what variables are direct causes or influences on what other ones, and builds local parent/child graphs

that way. It is usually very easy to identify where in such a structure the new variable goes, but one must be very careful to check for possible induced dependencies downstream.
$IcyWeather$ is not caused by any of the car-related variables, so needs no parents. It directly affects the battery and starter moter. $StarterMotor$ is an additional precondition for $Starts$. The new network is shown below:



(b) Give reasonable conditional probability tables for all the nodes.

Solution: (<u>Note:</u> this is a partial solution)

1. A reasonable prior for $IcyWeather$ might be 0.05 (perhaps, depending on the location and season).
2. $P(Battery \mid IcyWeather) = 0.95$,
   $P(Battery \mid \neg IcyWeather) = 0.997$
3. $P(StarterMotor \mid IcyWeather) = ...$,
   $P(StarterMotor \mid \neg IcyWeather) = ...$
4. $P(Radio \mid Battery) = ...$,
   $P(Radio \mid \neg Battery) = ...$
5. $P(Ignition \mid Battery) = 0.998$,
   $P(Ignition \mid \neg Battery) = 0.01$
6. $P(Gas) = 0.995$
7. $P(Starts \mid Ignition, StarterMotor, Gas) = ...$
8. $P(Moves \mid Starts) = ...$

There are more entries in the full CPT, however they have not been listed here for space reasons.

(c) How many independent values are contained in the joint probability distribution for eight Boolean nodes, assuming that no conditional independence relations are known to hold among them?

Solution:
With 8 Boolean variables, the joint has $2^8 - 1 = 255$ independent entries. In general there are 2^n entries in a joint probability distribution; however, only $2^n - 1$ *independent* entries. If the probabilities of all of the independent entries are summed up, the probability of the final entry must be $1 - \sum P(independent)$, making its value *dependent* on the rest.

(d) How many independent probability values do your network tables contain?

Solution: Your turn!

(e) **(optional)** The conditional distribution for $Starts$ could be described as a **noisy-AND** distribution. Define this family in general and relate it to the **noisy-OR** distribution.

Solution: (Note: this is a partial solution)

The CPT for $Starts$ describes a set of nearly necessary conditions that are together almost sufficient. That is, all the entries are nearly zero except for the entry where all conditions are true. That entry will be not quite 1 (because there is always some other possible fault that we didn't think of), but as we add more conditions it gets closer to 1. If we add a $Leak$ node as an extra parent, then the probability is exactly 1 when all parents are true. We can relate noisy-AND to noisy-OR using de Morgan's rule: $A \wedge B \equiv \neg(\neg A \vee \neg B)$. That is, noisy-AND is the same as noisy-OR except that the polarities of the parent and child variables are reversed. In the noisy-OR case, we have

$$P(Y = true \mid x_1, \ldots, x_k) = 1 - \prod_{\{i:x_i = true\}} q_i \tag{1}$$

where q_i is the probability that the *presence* of the *i*th parent *fails* to cause the child to be *true*. In the noisy-AND case, we can write

$$P(Y = true \mid x_1, \ldots, x_k) = 1 - .... \tag{2}$$

where r_i is the probability that the *absence* of the *i*th parent *fails* to cause the child to be *false* (e.g., it is magically bypassed by some other mechanism).

(f) Reconstruct, showing your workings, five entries of the full joint probability distribution (e.g., reconstruct $P(\neg iW, b, r, i, g, s, sM, \neg m)$ or $P(iW, \neg b, r, i, \neg g, \neg s, sM, m)$, or any other variable assignment).

Solution: (Note: this is a partial solution)

Here is one example. The probabilities are taken from question 1(b).

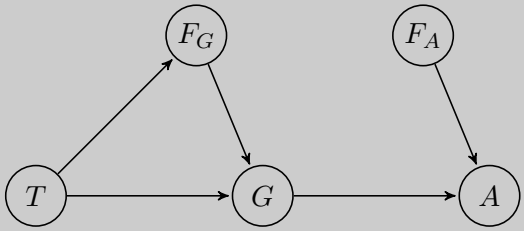
$$\begin{aligned}
&P(\neg iW, b, r, i, g, s, sM, \neg m) = \\
&P(\neg iW) \cdot P(b \mid \neg iW) \cdot P(r \mid b) \cdot P(i \mid b) \cdot P(g) \cdot P(s \mid i, sM, g) \cdot .... = \\
&(1 - P(iW)) \cdot P(b \mid \neg iW) \cdot P(r \mid b) \cdot P(i \mid b) \cdot .... = \\
&0.95 \cdot 0.997 \cdot 0.9999 \cdot 0.998 \cdot 0.995 \cdot 0.9999 \cdot 0.999 \cdot 0.002 = 0.00188
\end{aligned}$$

PART 3: Power station diagnosis

In your local nuclear power station, there is an alarm that senses when a temperature gauge exceeds a given threshold. The gauge measures the temperature of the core. Consider the Boolean variable A (alarm sounds), F_A (alarm is faulty), and F_G (gauge is faulty) and the multivalued nodes G (gauge reading) and T (actual core temperature).

(a) Draw a Bayesian network for this domain, given that the gauge is more likely to fail when the core temperature gets too high.

Solution:
A suitable network is shown in the figure below. The key aspects are: the failure nodes are parents of the sensor nodes, and the temperature node is a parent of both the gauge and the gauge failure node. It is exactly this kind of correlation that makes it difficult for humans to understand what is happening in complex systems with unreliable sensors.



(b) Is your network a polytree?

Solution: Your turn!

(c) Suppose there are just two possible actual and measured temperatures, normal and high; the probability that the gauge gives the correct temperature is x when it is working, but y when it is faulty. Give the conditional probability table associated with G.

Solution: (Note: this is a partial solution)

The CPT for G is shown below. The wording of the question is a little tricky because x and y are defined in terms of "incorrect" rather than "correct".

	$T = Normal$		$T = High$	
	F_G	$\neg F_G$	F_G	$\neg F_G$
$G = High$	$1 - y$	$1 - x$	....	....
$G = Normal$	y	x	....	....

(d) Suppose the alarm works correctly unless it is faulty, in which case it never sounds. Give the conditional probability table associated with A.

Solution: (Note: this is a partial solution)

The alarm works correctly unless it is faulty. So, if the alarm is not faulty, then the alarm works correctly, meaning that it sounds only if gauge reading is high.
The CPT for A is as follows:

	$G = Normal$		$G = High$	
	F_A	$\neg F_A$	F_A	$\neg F_A$
A	0	0	0	1
$\neg A$	....	....	....	....

(e) **(optional)** Suppose the alarm and gauge are working and the alarm sounds. Using the information from the previous questions, calculate an expression for the probability that the temperature of the core is too high, in terms of the various conditional probabilities in the network.

Solution: (Note: this is a partial solution)

The assumption says that the alarm and gauge are working, which means that $F_A = \neg f_a$ and $F_G = \neg f_g$.

Also, we have that the alarm sounds $(A = a)$. As we are asked the probability of the temperature being too high $(T = t)$, we can formulate the query as:

$$P(t \mid a, \neg f_a, \neg f_g)$$

Which we would have to evaluate for the values of G (the remaining variable in our network). However, since we are using the assumptions above (also from parts (c) and (d)) we can deduce the following: Since the alarm sounds (a) and the alarm is not faulty $(\neg f_a)$, we can infer by the table above that for this to be possible, the gauge reading must be high $(G = g)$. Hence, we can rewrite the above expression as:

$$P(t \mid a, \neg f_a, g, \neg f_g) = \cdots$$

Your turn!

PART 4: Bayesian Networks via pgmpy .....

Finally, let's get our hands dirty with Bayesian Networks in Python!

One can build and do inference with Bayesian Networks in Python using the pgmy package!

So, head to pgmy home page and:

(a) Install and set-up the package in your machine.

(b) Understand and run the Earthquake example in the documentation.

(c) Use the system to perform and validate some of the inferences you did in Part 1 for this domain.

(d) **Enjoy & learn Bayesian Networks with Python!** Make sure you understand what the code is doing and that it does not appear as "magic" to you!

End of tutorial worksheet

MDP Cheatsheet

1. Bellman Equations

- State Value (Policy Evaluation):
  $U^{\pi}(s) = R(s) + \gamma \Sigma_{s'} T(s, \pi(s), s') U^{\pi}(s')$
- Action Value (Q-function):
  $Q(s,a) = R(s) + \gamma \Sigma_{s'} T(s,a,s') U(s')$
- Optimal Value (Value Iteration):
  $U(s) = R(s) + \gamma \max_a \Sigma_{s'} T(s,a,s') U(s')$

2. Algorithms

- Policy Evaluation: Compute $U^{\pi}(s)$ for fixed π.
- Policy Improvement: $\pi'(s) = \text{argmax}_a \Sigma_{s'} T(s,a,s') U^{\pi}(s')$
- Policy Iteration: Alternate Policy Evaluation + Improvement.
- Value Iteration: Iteratively apply optimal Bellman update (includes max).

3. Usage

- Policy Evaluation $\rightarrow$ Evaluate quality of a given policy.
- Policy Iteration $\rightarrow$ Find optimal policy by evaluation + improvement.
- Value Iteration $\rightarrow$ Directly compute optimal utilities & π^*.

4. MEU (Maximum Expected Utility)

$\pi^*(s) = \text{argmax}_a \Sigma_{s'} T(s,a,s') U(s')$

Worked Example (2x2 Grid)

- States: (1,1)=+10 (terminal), (1,2)=-1, (2,1)=-1, (2,2)=-1
- Actions: N, E, S, W (deterministic).
- Discount: γ=0.9.

Iteration 0 (init U=0):

U(1,2)= -1 + 0.9 * max{ U(1,1)=0, U(2,2)=0 } = -1.
U(2,1)= -1 + 0.9 * max{ U(1,1)=0, U(2,2)=0 } = -1.
U(2,2)= -1 + 0.9 * max{ U(2,1)=0, U(1,2)=0 } = -1.

Iteration 1:

U(1,2)= -1 + 0.9 * U(1,1)= -1 + 0.9*10 = 8.
U(2,1)= -1 + 0.9 * U(1,1)= -1 + 9 = 8.
U(2,2)= -1 + 0.9 * max{ U(2,1), U(1,2) }= -1 + 0.9*8 = 6.2.

Result:

- Optimal values: U(1,2)=8, U(2,1)=8, U(2,2)=6.2.
- Optimal policy π^*: move toward (1,1).